数字信号分析与处理

程序设计报告

完成日期：2026 年 6 月 24 日

一、 基本离散序列的 MATLAB 程序实现

1. 用 MATLAB 程序实现如下正（余）弦序列，并画出图形(用 stem 函数画图)。(每小题 1 分，共 8 分)

1) $x_3(n) = 2\sin\left(\frac{2\pi n}{20}\right),\ -25 \le n \le 25$

2) $x_4(n) = 2\sin\left(\frac{2\pi n}{\sqrt{228}}\right),\ 0 \le n \le 50$

3) $x_5(n) = 2\cos\left(\frac{2\pi n}{\sqrt{228}} - \frac{\pi}{2}\right),\ 0 \le n \le 50$

4) $x_6(n) = 2\cos\left(\frac{2\pi n}{\sqrt{228}} + \frac{3\pi}{2}\right),\ 0 \le n \le 50$（注意观察$x_4, x_5, x_6$波形）

5) $x_7(n) = 2\sin\left(\frac{2\pi n}{3.5^{2.8}}\right),\ 0 \le n \le 50$

(以下三题提示：for 循环或 .* ./)

6) $x_8(n) = 0.2n\sin\left(\frac{2\pi n}{3.5^{2.8}}\right),\ 0 \le n \le 100$

(axis([0,100,-20,20]);)

7) $x_9(n) = 2\sin\left(\frac{2\pi n}{(0.2n+1.5^{2.8})}\right),\ 0 \le n \le 100$

(axis([0,100,-3,3]);)

8) $x_{10}(n) = (2 + \frac{50}{n+1})\sin\left(\frac{2\pi n}{(0.2n+1.5^{2.8})}\right),\ 0 \le n \le 100$

(axis([0,100,-30,30]);)

【原理与编程思路】

离散正 (余) 弦序列可统一写为 x[n]=A sin(ω0n+φ) 或 x[n]=A cos(ω0n+φ), 其中 A 决定振幅，ω0 决定相位随 n 的变化速度，φ 决定初相位。离散正弦序列只有在 ω0/(2π) 为有理数时才存在整数周期；因此 sqrt(228) 等非整数周期参数应重点观察其离散采样后的重复性。x4、x5、x6 可利用 cos(θ-π/2)=sinθ 以及相位相差 2π 的等价性判断波形关系。x8 的 0.2n 是线性增长包络，x9 的分母随 n 增大使相位增长逐渐变慢，x10 同时含有衰减包络和非线性相位。

编程时先建立 n 的取值向量，再按表达式逐点计算。向量之间按元素相乘或相除时必须使用 .* 和 ./，否则 MATLAB 会把运算解释为矩阵运算。必要时设置简短中间量保存周期、相位或包络，最后使用 stem 绘制序列并用 axis 固定显示范围。

【编程代码及详细注释】

注释说明：代码只在算法选择、公式对应关系和边界处理等关键位置进行说明，常规赋值、绘图和变量初始化不再逐行注释。

Q1_01.m

```
% Q1_01.m
% 功能：产生并绘制正弦序列 x3(n)=2sin(2*pi*n/20)。
% 原理：幅值为 2，周期参数为 20；对给定的 n 逐点代入表达式即可。

clear;
clc;
close all;

n = -25:0.01:25;
x = 2*sin(2*pi*n/20);

figure;
stem(n,x);
xlabel('n');
ylabel('x3(n)');
title('x3(n)=2*sin(2*pi*n/20)');
axis([-25,25,-3,3]);
grid on;
```

Q1_02.m

```
% Q1_02.m
% 功能：产生并绘制正弦序列 x4(n)=2sin(2*pi*n/sqrt(228))。
% 说明：sqrt(228) 不是整数，因此在整数 n 上该离散正弦序列不具有整数周期。

clear;
clc;
close all;

n = 0:0.01:50;
T = sqrt(228);
x = 2*sin(2*pi*n/T);

figure;
stem(n,x);
xlabel('n');
ylabel('x4(n)');
title('x4(n)=2*sin(2*pi*n/sqrt(228))');
```

```
axis([0,50,-3,3]);
grid on;
```

Q1_03.m

```
% Q1_03.m
% 功能：绘制带初相位 -pi/2 的余弦序列。
% 原理：由 cos(theta-pi/2)=sin(theta)，该序列应与 Q1_02 的波形相同。

clear;
clc;
close all;

n = 0:0.01:50;
T = sqrt(228);
p = -pi/2;
x = 2*cos(2*pi*n/T + p);

figure;
stem(n,x);
xlabel('n');
ylabel('x5(n)');
title('x5(n)=2*cos(2*pi*n/sqrt(228)-pi/2)');
axis([0,50,-3,3]);
grid on;
```

Q1_04.m

```
% Q1_04.m
% 功能：绘制初相位为 3*pi/2 的余弦序列。
% 原理：相位相差 2*pi 的正弦/余弦表达式等价，因此本题波形也与 x4(n) 相同。

clear;
clc;
close all;

n = 0:0.01:50;
T = sqrt(228);
p = 3*pi/2;
x = 2*cos(2*pi*n/T + p);

figure;
stem(n,x);
xlabel('n');
ylabel('x6(n)');
title('x6(n)=2*cos(2*pi*n/sqrt(228)+3*pi/2)');
```

```
axis([0,50,-3,3]);
grid on;
```

Q1_05.m

```
% Q1_05.m
% 功能：绘制周期参数为 3.5^2.8 的正弦序列。
% 思路：先计算常量周期参数，再把整个 n 向量代入正弦表达式。

clear;
clc;
close all;

n = 0:0.01:50;
T = 3.5^2.8;
x = 2*sin(2*pi*n/T);

figure;
stem(n,x);
xlabel('n');
ylabel('x7(n)');
title('x7(n)=2*sin(2*pi*n/3.5^2.8)');
axis([0,50,-3,3]);
grid on;
```

Q1_06.m

```
% Q1_06.m
% 功能：绘制振幅随 n 线性增大的调幅正弦序列。
% 原理：0.2*n 是包络，正弦项是载波；二者必须使用 .* 逐元素相乘。

clear;
clc;
close all;

n = 0:0.01:100;
T = 3.5^2.8;
a = 0.2*n;
c = sin(2*pi*n/T);
x = a.*c;

figure;
stem(n,x);
xlabel('n');
ylabel('x8(n)');
title('x8(n)=0.2*n.*sin(2*pi*n/3.5^2.8)');
```

```
axis([0,100,-20,20]);
grid on;
```

Q1_07.m

```
% Q1_07.m
% 功能：绘制相位为非线性函数的变频正弦序列。
% 原理：分母 0.2*n+1.5^2.8 随 n 增大，使相位增长速度逐渐变慢，
%       因而相邻振荡之间的间隔逐渐增大，表现为瞬时频率降低。

clear;
clc;
close all;

n = 0:0.01:100;
d = 0.2*n + 1.5^2.8;
p = 2*pi*n./d;
x = 2*sin(p);

figure;
stem(n,x);
xlabel('n');
ylabel('x9(n)');
title('x9(n)=2*sin(2*pi*n./(0.2*n+1.5^2.8))');
axis([0,100,-3,3]);
grid on;
```

Q1_08.m

```
% Q1_08.m
% 功能：绘制同时具有时变振幅和时变频率的正弦序列。
% 原理：2+50/(n+1) 构成衰减包络；相位项与 Q1_07 相同。
%       n+1 可避免 n=0 时发生除零。

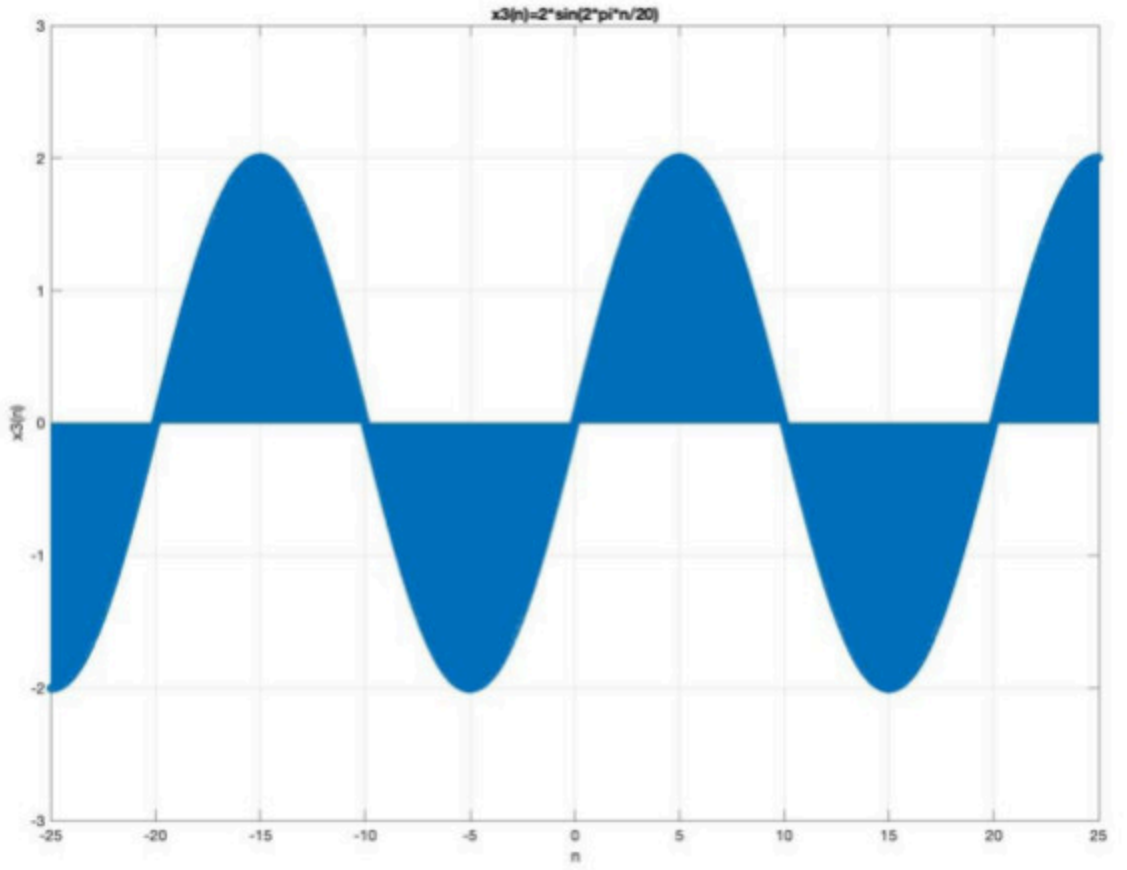
clear;
clc;
close all;

n = 0:0.01:100;
a = 2 + 50./(n+1);
p = 2*pi*n./(0.2*n+1.5^2.8);
x = a.*sin(p);

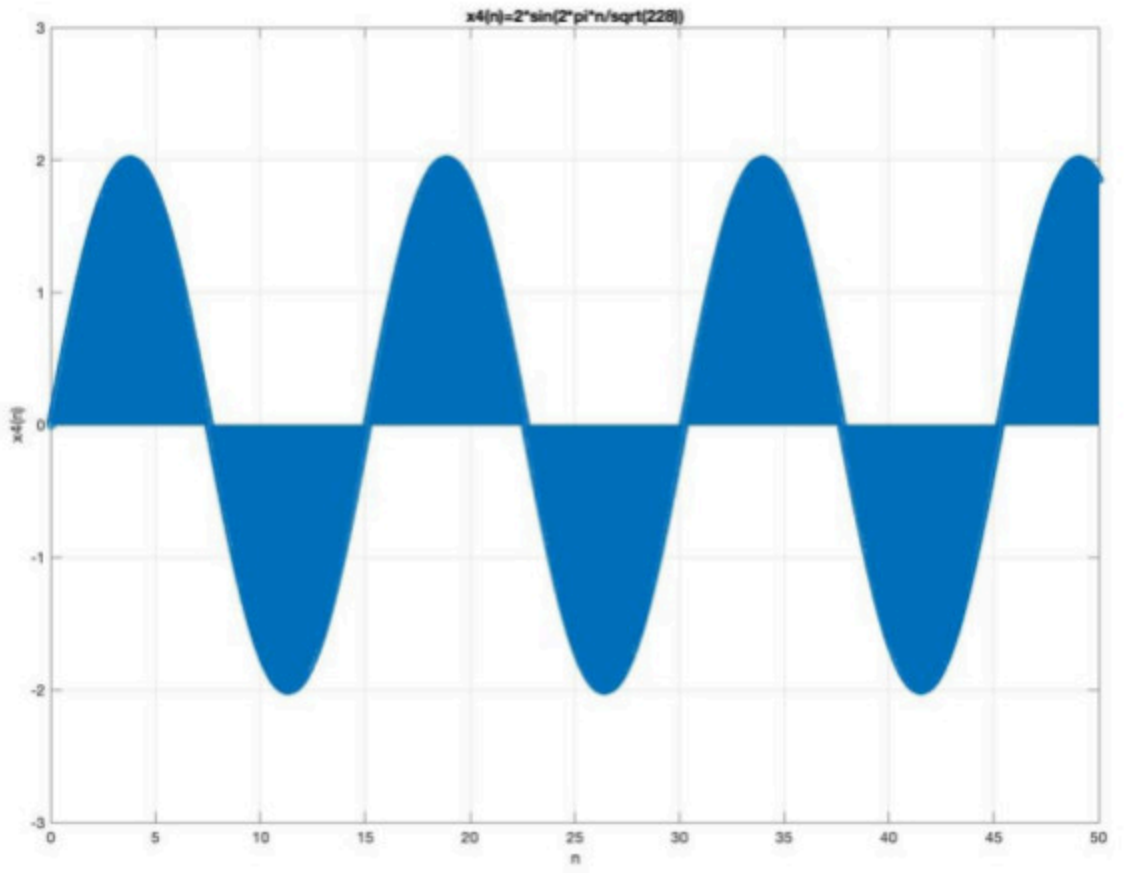
figure;
stem(n,x);
xlabel('n');
```

```
ylabel('x10(n)');
title('x10(n)=(2+50./(n+1)).*sin(2*pi*n./(0.2*n+1.5^2.8))');
axis([0,100,-30,30]);
grid on;
```

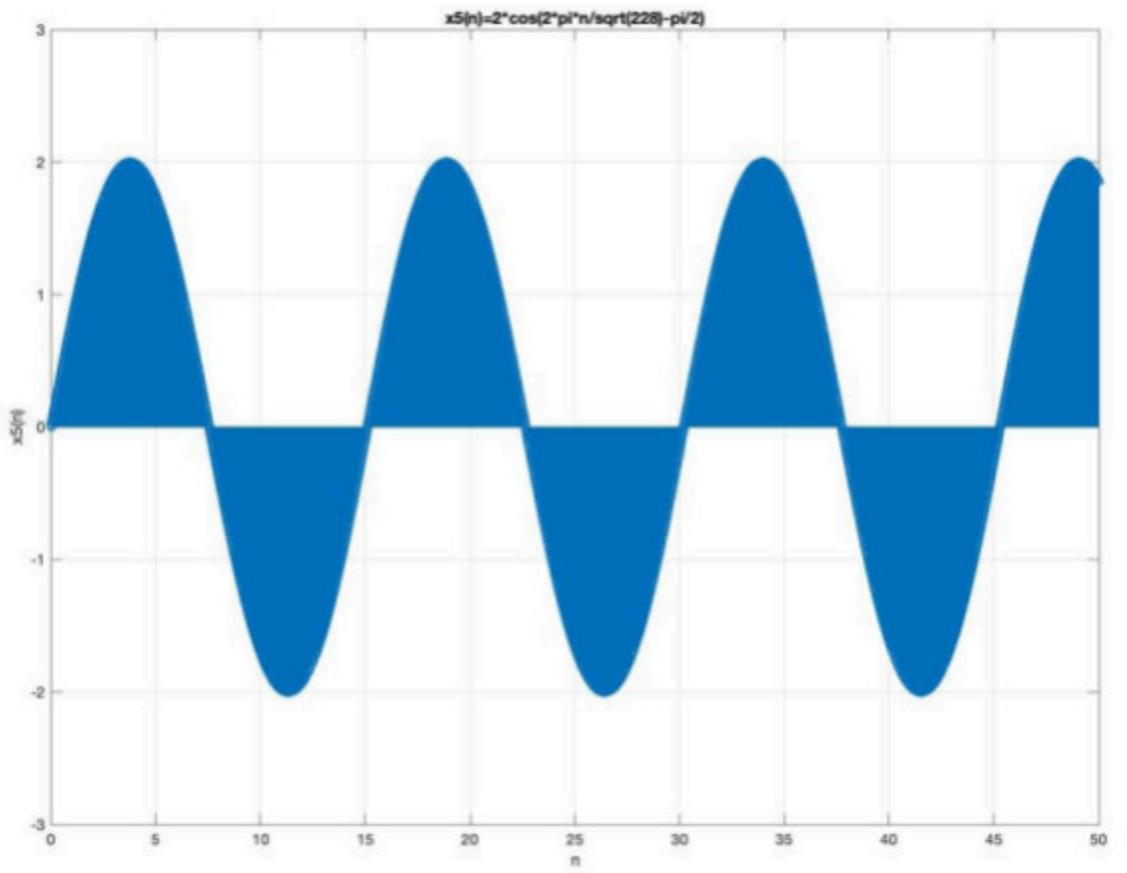
【运行结果】



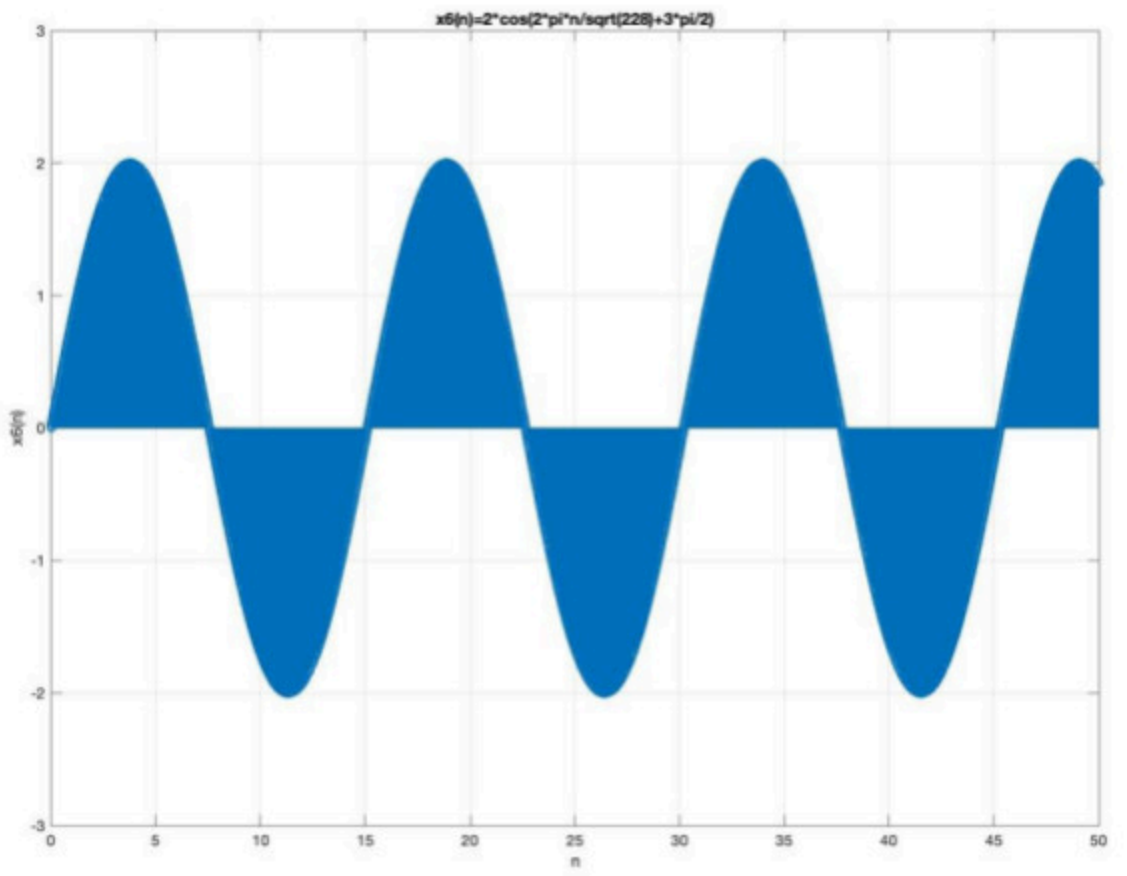
x3(n)



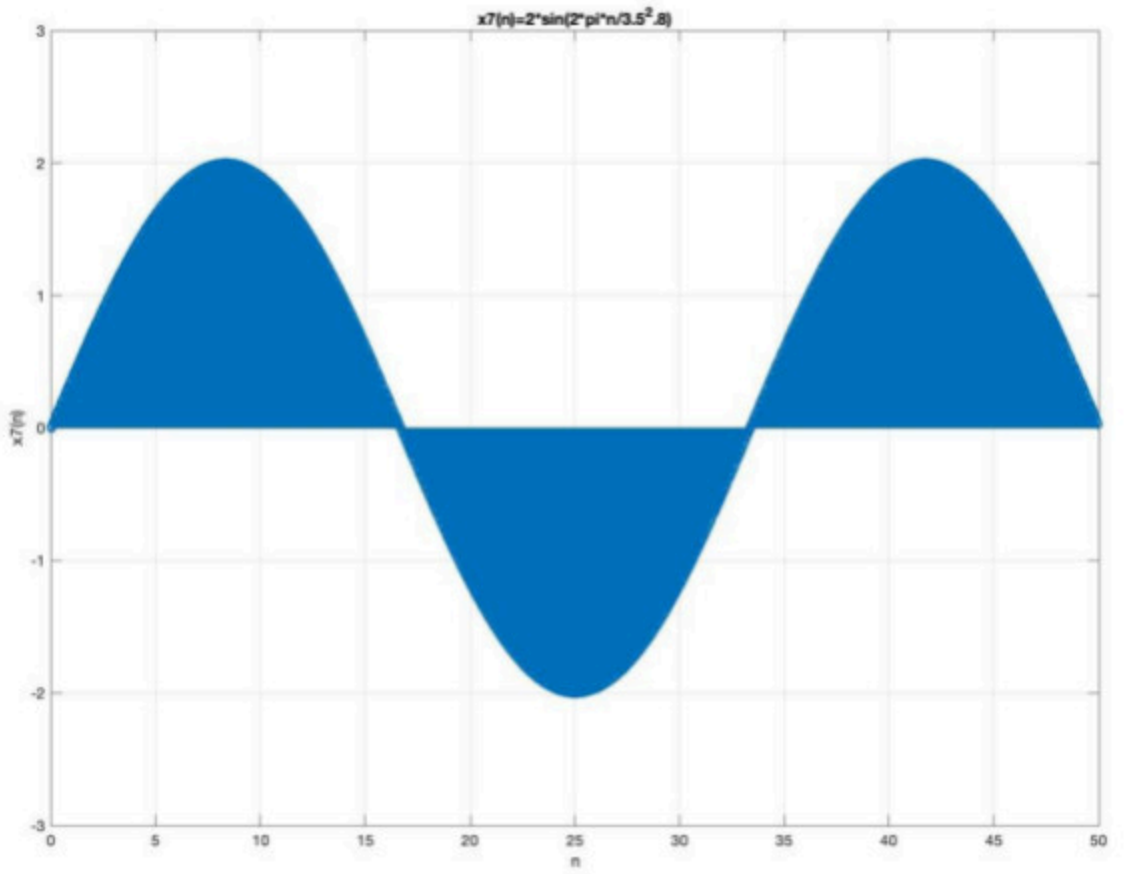
x4(n)


x5(n)=2*cos(2*pi*n/sqrt(228)-pi/2)
x5(n)
n

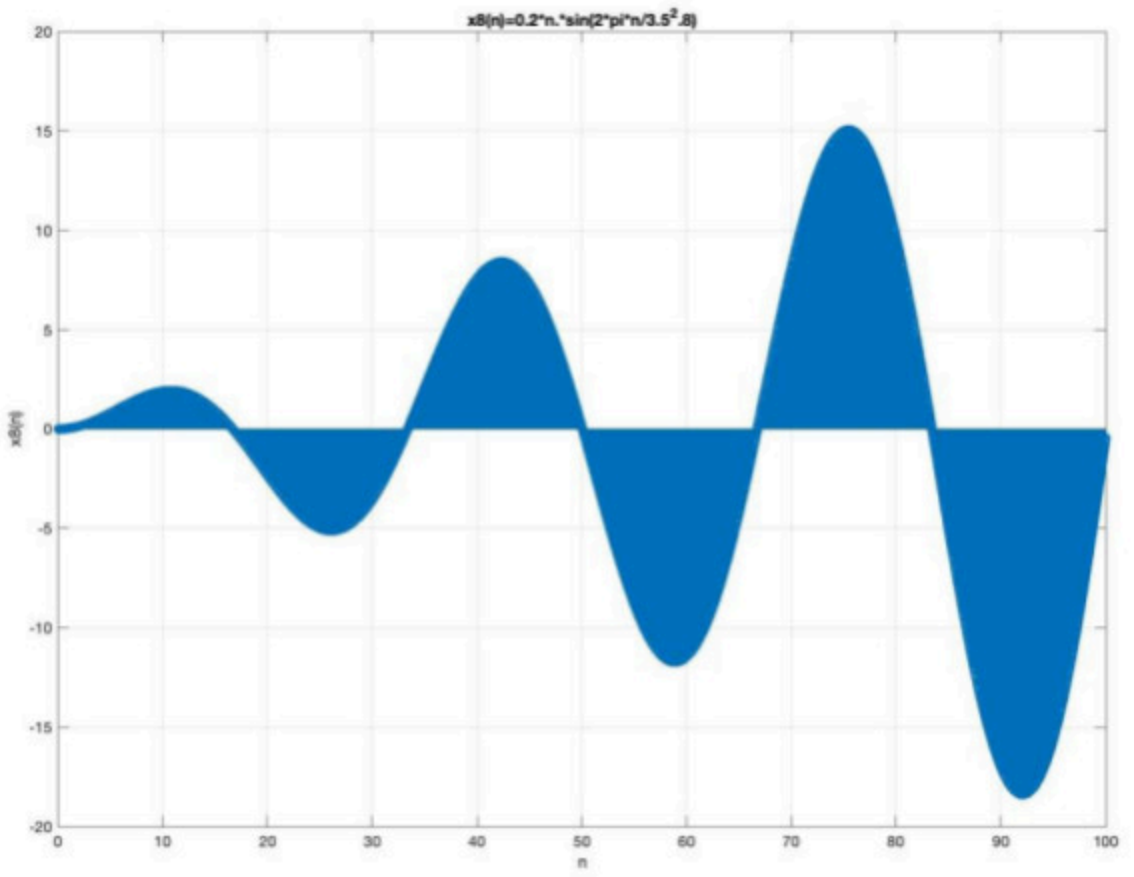
x5(n)


x6(n)=2*cos(2*pi*n/sqrt(228)+3*pi/2)
x6(n)
n

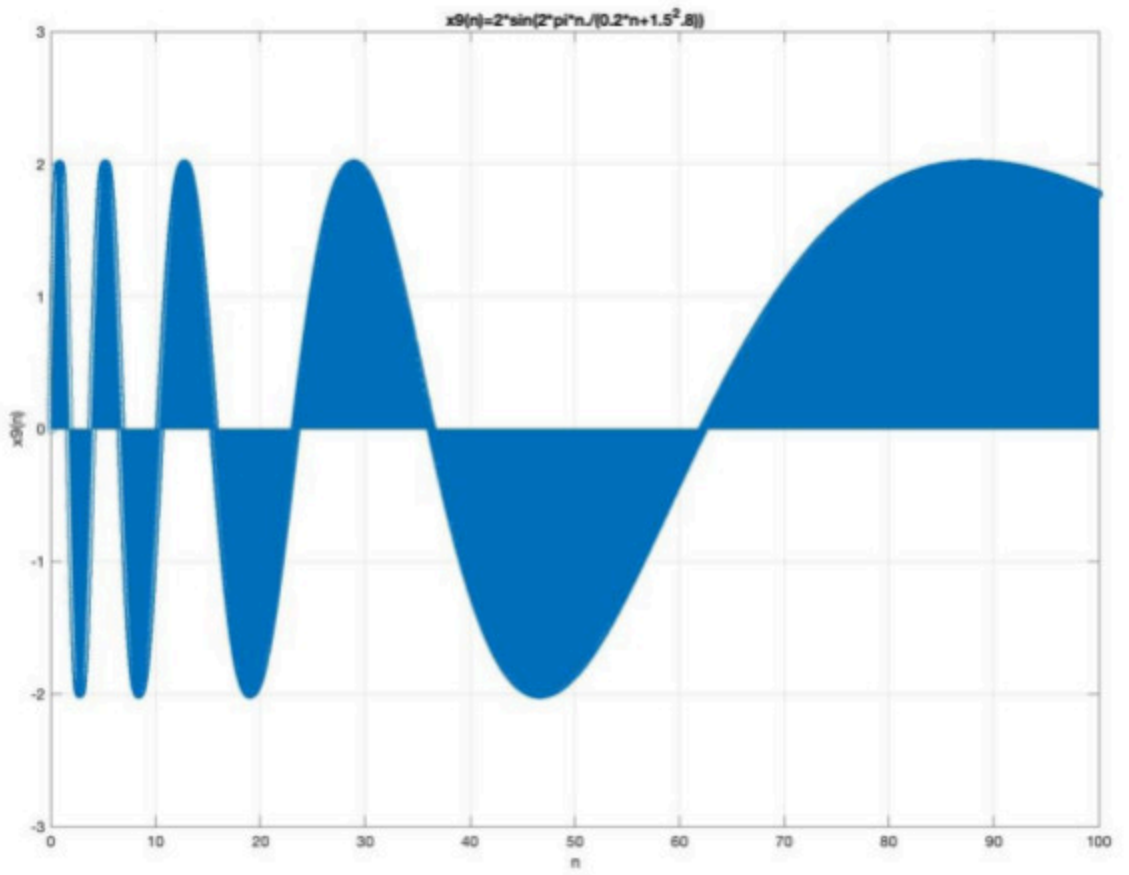
x6(n)



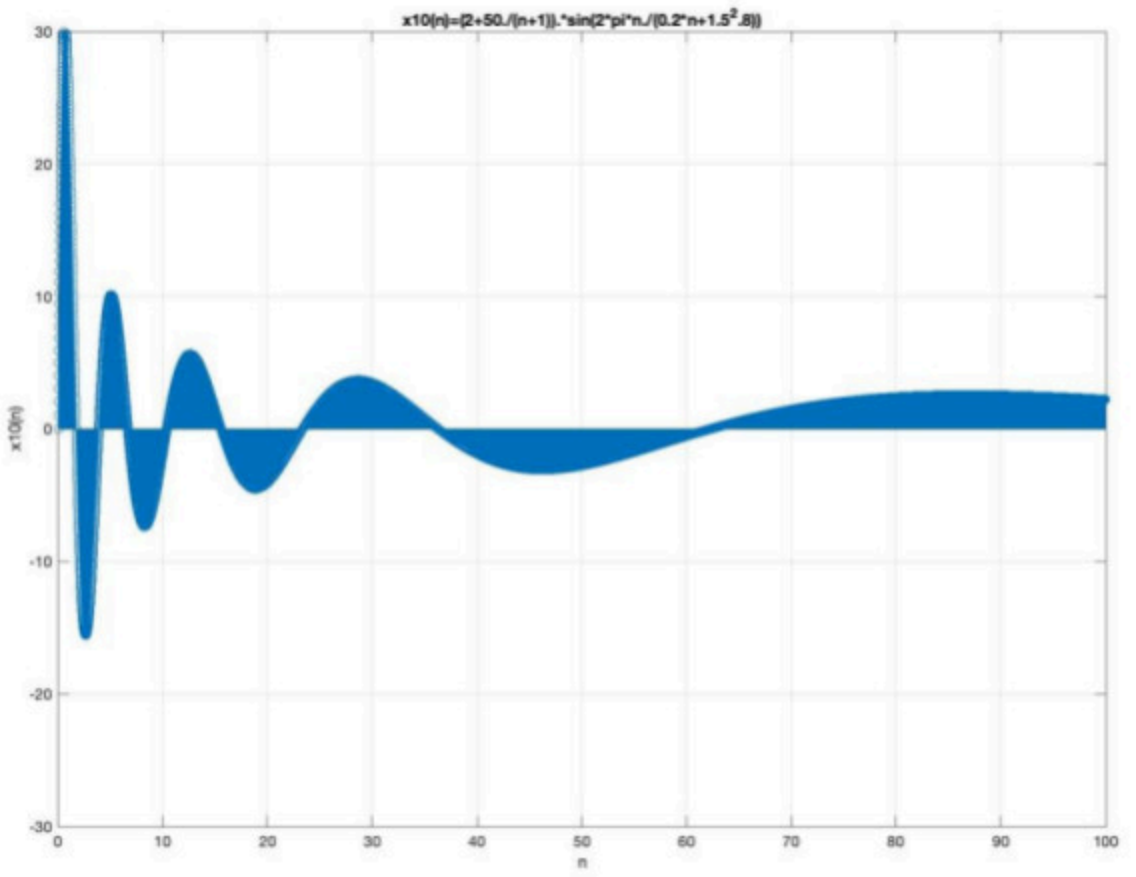
x7(n)



x8(n)



x9(n)



x10(n)

从图中可以看出，x4、x5 和 x6 的波形相同，说明它们虽然采用正弦或余弦表示，但相位变换后是等价的。x8 的振幅随 n 增大，x9 的瞬时频率逐渐降低，x10 的振幅总体随 n 减小。

2. 用 MATLAB 程序实现如下冲激、阶跃、矩形序列（要求写成 function 形式）。（每小题 2 分，共 6 分）

1) $\delta(n-n_0)=\begin{cases}1, & n=n_0\\ 0, & else\end{cases}, \quad n_1\le n\le n_2$

(提示：输入参数为n_0, n_1, n_2)

2) $u(n-n_0)=\begin{cases}1, & n\ge n_0\\ 0, & n<n_0\end{cases}, \quad n_1\le n\le n_2$

(提示：输入参数为n_0, n_1, n_2)

3) $r_N(n-n_0)=\begin{cases}1, & n_0\le n\le n_0+N-1\\ 0, & else\end{cases}, \quad n_1\le n\le n_2$

(提示：输入参数为n_0, N, n_1, n_2)

【原理与编程总路】

单位冲激序列 δ[n-n0] 只在 n=n0 处取 1，其余位置为 0；单位阶跃序列 u[n-n0] 在 n<n0 时为 0、n≥n0 时为 1；宽度为 N 的矩形序列在 n0≤n≤n0+N-1 内取 1，因此恰好包含 N 个非零样点。这三个序列都是构造复杂离散信号的基本模块。

函数首先由 n1:n2 建立统一的输出下标，并预分配全零数组。随后使用 for 循环逐点访问 n(i)，通过 if 判断当前样点是否满足非零条件。函数同时返回 x 和 n，调用者可以直接进行线性组合和绘图。矩形序列右端点写成 n0+N-1，是为了避免把端点多计算一个样点。

【编程代码及详细注释】

```
function [x,n] = zzh_impulse(n0,n1,n2)
% 生成有限观察区间内的单位冲激序列。
% 输入：n0 为冲激位置；n1、n2 为输出序列的起止下标。
% 输出：x 为冲激序列，n 为对应的离散下标。


n = n1:n2;
x = zeros(1,length(n));


for i = 1:length(n)
    if n(i) == n0
        x(i) = 1;
```

```
    else
        x(i) = 0;
    end
end
end

function [x,n] = zzh_jump(n0,n1,n2)
% 生成有限观察区间内的单位阶跃序列。
% 输入：n0 为阶跃起点；n1、n2 为输出下标范围。
% 定义：当 n<n0 时 u(n-n0)=0，当 n>=n0 时 u(n-n0)=1。

n = n1:n2;
x = zeros(1,length(n));

for i = 1:length(n)
    if n(i) >= n0
        x(i) = 1;
    else
        x(i) = 0;
    end
end
end

function [x,n] = zzh_rectangle(n0,N,n1,n2)
% 生成宽度为 N 的离散矩形序列。
% 输入：n0 为矩形起点，N 为非零样点数，n1:n2 为观察区间。
% 非零范围：n0 <= n <= n0+N-1，共包含 N 个离散样点。

n = n1:n2;
x = zeros(1,length(n));

for i = 1:length(n)
    if n(i) >= n0 && n(i) <= n0+N-1
        x(i) = 1;
    else
        x(i) = 0;
    end
end
end
```

3. 调用上述三个子函数，实现如下序列，并画出图形(用 stem 函数画图)。(每小题 2 分，共 8 分)

   1) $x_1(n) = \delta(n-2) - 2\delta(n+5), \quad -10 \le n \le 10$

      (axis([-10,10,-3,3]))

2) $x_2(n) = -u(n+10) + 2u(n-3), \quad -10 \le n \le 10$

(axis([-10,10,-3,3]))

3) $x_3(n) = r_{11}(n+5) - r_5(n+2), \quad -10 \le n \le 10$

(axis([-10,10,-3,3]))

4) $x_4(n) = u(n+10) - r_3(n+5) - r_3(n-3), \quad -10 \le n \le 10$

(axis([-10,10,-3,3]))

【编程思路】

离散系统中的序列可以通过基本序列的移位、加权和叠加来表示。冲激项直接决定孤立非零点的位置与幅值；阶跃之差可形成有限区间；矩形序列则可直接描述连续若干个离散样点。由于各子函数都在相同的 n=-10:10 下返回结果，因此可以按题目系数进行逐点加减。程序分别为每一个移位项调用 zzh_impulse、zzh_jump 或 zzh_rectangle，并使用简短变量保存中间序列。完成线性组合后，每个结果分别用单独的 figure 绘制。验证时重点检查冲激位置、阶跃发生位置、矩形区间以及叠加后各区间的幅值是否与解析式一致。

【编程代码及详细注释：Q1_09.m】

```
% Q1_09.m
% 功能：调用冲激、阶跃和矩形序列函数，构造四个组合离散序列。
% 思路：先生成各基本序列，再按照题目给出的系数进行逐点线性组合。


clear;
clc;
close all;


% x1(n)=delta(n-2)-2delta(n+5)
[d2,n] = zzh_impulse(2,-10,10);
dm5 = zzh_impulse(-5,-10,10);
x1 = d2 - 2*dm5;

% x2(n)=-u(n+10)+2u(n-3)
um10 = zzh_jump(-10,-10,10);
u3 = zzh_jump(3,-10,10);
x2 = -um10 + 2*u3;

% x3(n)=r11(n+5)-r5(n+2)
r11 = zzh_rectangle(-5,11,-10,10);
```

```
r5 = zzh_rectangle(-2,5,-10,10);
x3 = r11 - r5;

% x4(n)=u(n+10)-r3(n+5)-r3(n-3)
u0 = zzh_jump(-10,-10,10);
rl = zzh_rectangle(-5,3,-10,10);
rr = zzh_rectangle(3,3,-10,10);
x4 = u0 - rl - rr;

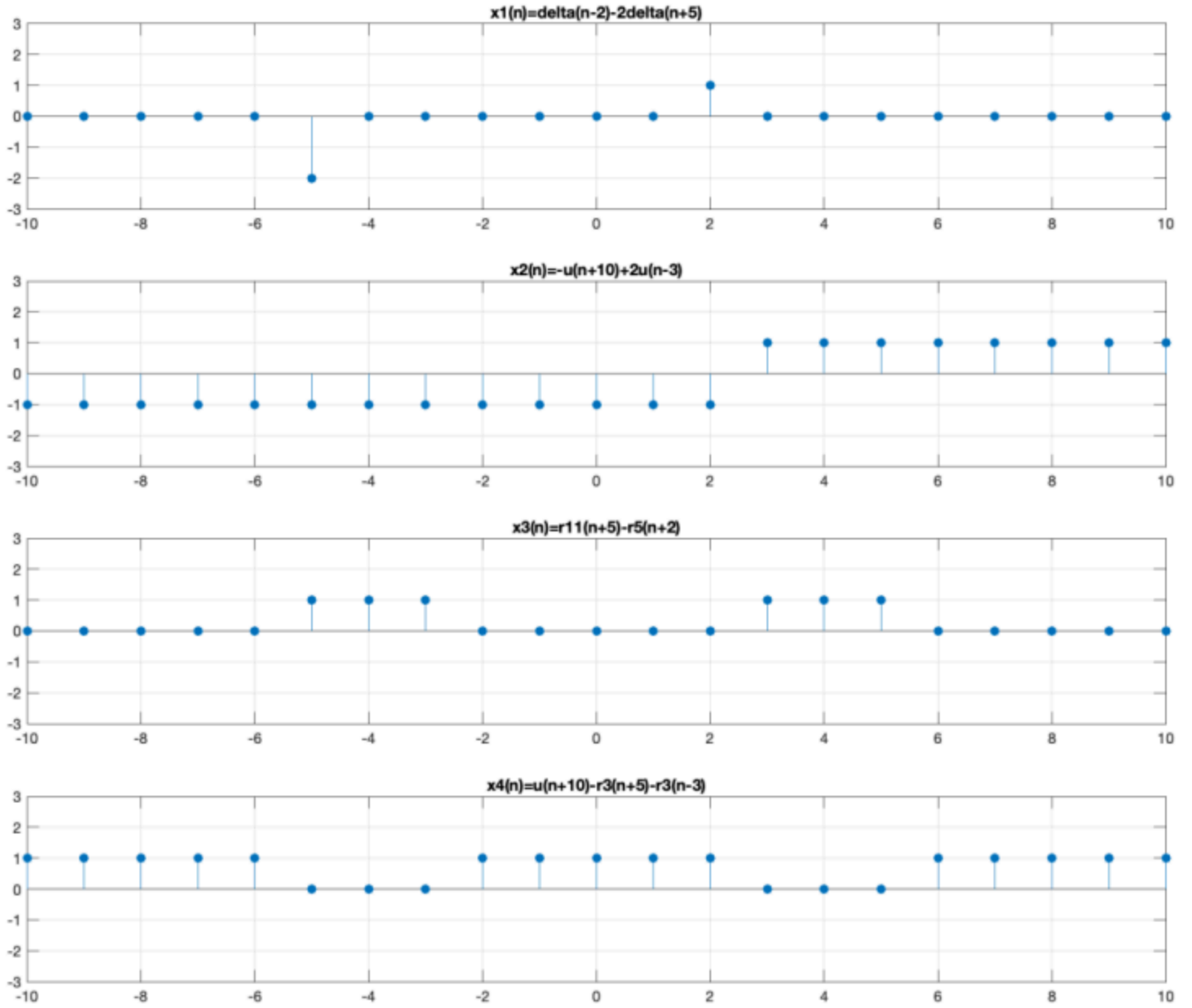
figure;
stem(n,x1,'filled');
title('x1(n)=delta(n-2)-2delta(n+5)');
axis([-10,10,-3,3]);
grid on;

figure;
stem(n,x2,'filled');
title('x2(n)=-u(n+10)+2u(n-3)');
axis([-10,10,-3,3]);
grid on;

figure;
stem(n,x3,'filled');
title('x3(n)=r11(n+5)-r5(n+2)');
axis([-10,10,-3,3]);
grid on;

figure;
stem(n,x4,'filled');
title('x4(n)=u(n+10)-r3(n+5)-r3(n-3)');
axis([-10,10,-3,3]);
grid on;
```

【运行结果】



四个组合序列的运行结果

图中各非零区间与理论表达式一致。例如 x1 只在 n=-5 和 n=2 处非零，x2 在 n<3 时为 -1、n≥3 时为 1。

二、 离散序列褶积与相关运算的 MATLAB 程序实现

1. 用 MATLAB 程序分别实现（要求写成 function 形式）两个序列的线性褶积、循环褶积、线性相关、循环相关．命名规则如下．(每小题 3 分．共 12 分)

```
close all
clear
clc
x=magic(100)-mean(magic(100));
```

2. 分析循环褶积的长度应满足什么条件时，所得到的线性褶积与循环褶积在有效离散数值上是对等的？并利用所编写的子函数展示这一性质．(2 分)
3. 分析如何利用线性褶积运算来实现线性相关？并利用所编写的子函数展示这一性质．(2 分)

【原理与编程思路】

长度分别为 N1、N2 的有限序列，其线性褶积定义为 $y[i]=\Sigma x1[j]x2[i-j]$，完整输出长度为 N1+N2-1．程序固定输出下标 i，再遍历 x1 的下标 j，并仅在 x2 对应下标有效时累加。N 点循环褶积把序列视为以 N 为周期延拓，负下标通过加 N 回绕到 0～N-1．若 $N\geq N1+N2-1$，补零后的循环褶积不会发生时域混叠，因而与线性褶积相同；若 N 较小，线性褶积尾部会折叠到前部。

对实序列，线性互相关可以写成 x1 与 x2 反序序列的线性褶积；若处理复序列，还需要对 x2 取复共轭。循环相关则对每一个循环位移 m 累加 $x1[i]x2[(i-m) \bmod N]$。代码全部按定义使用循环实现，并通过 Q2_02.m 的曲线对比展示循环褶积条件和相关—褶积关系。

【编程代码及详细注释】

zzh_lineconv.m

```
function y = zzh_lineconv(x1,x2)
% 按定义计算两个有限长序列的线性褶积。
```

```
% 输入：x1、x2 为行向量或可用 length 访问的一维序列。
% 输出：y 的长度为 N1+N2-1。
% 定义：y(i) 为所有有效 x1(j)*x2(i-j+1) 之和，越界项不参与累加。

N1 = length(x1);
N2 = length(x2);
y = zeros(1,N1+N2-1);

for i = 1:N1+N2-1
    for j = 1:N1
        k = i-j+1;

        if k>=1 && k<=N2
            y(i) = y(i)+x1(j)*x2(k);
        end
    end
end
end
```

zzh_circonv.m

```
function y = zzh_circonv(x1,x2,N)
% 按定义计算 N 点循环褶积。
% 输入：x1、x2 为待运算序列，N 为循环长度。
% 处理：先将两个序列补零到 N 点，再进行周期下标运算。
% 注意：若 N>=length(x1)+length(x2)-1，结果与线性褶积等价；
%       若 N 较短，则线性褶积结果会发生周期折叠（时域混叠）。

x1n = zeros(1,N);
x2n = zeros(1,N);
y = zeros(1,N);

for i = 1:N
    if i <= length(x1)
        x1n(i) = x1(i);
    end
    if i <= length(x2)
        x2n(i) = x2(i);
    end
end

% 负下标加 N，实现循环下标回绕。
for i = 0:N-1
    for j = 0:N-1
        k = i-j;
```

```
        if k < 0
            k = k+N;
        end
        y(i+1) = y(i+1)+x1n(j+1)*x2n(k+1);
    end
end
end
```

zzh_linecorr.m

```
function y = zzh_linecorr(x1,x2)
% 利用“反序后褶积”计算两个实序列的线性互相关。
% 对实序列：r12 = x1 * x2(−n)。
% 若输入为复序列，严格定义还应对反序后的 x2 取复共轭。

x2r = zeros(1,length(x2));
for i = 1:length(x2)
    x2r(i) = x2(length(x2)−i+1);
end

y = zzh_lineconv(x1,x2r);
end
```

zzh_circorr.m

```
function y = zzh_circorr(x1,x2,N)
% 按定义计算两个实序列的 N 点循环互相关。
% 对每个循环位移 m，累加 x1(i)*x2((i−m) mod N)。
% 输入不足 N 点时在尾部补零；本实现不调用内置 circshift 或 xcorr。

x1n = zeros(1,N);
x2n = zeros(1,N);
y = zeros(1,N);

for i = 1:N
    if i <= length(x1)
        x1n(i) = x1(i);
    end
    if i <= length(x2)
        x2n(i) = x2(i);
    end
end

% 对每个循环位移 m 计算一次相关和。
for m = 0:N−1
```

```
    for i = 0:N-1
        j = i-m;

        if j < 0
            j = j+N;
        end
        y(m+1) = y(m+1)+x1n(i+1)*x2n(j+1);
    end
end
end
```

Q2_01.m

```
% Q2_01.m
% 功能：调用自编函数，分别完成线性/循环褶积和线性/循环相关。
% 数据：magic(100) 的不同行作为测试序列；减去列均值以削弱直流分量。

close all;
clear;
clc;

raw = magic(100);
x = raw - mean(raw);

% 1. 线性褶积：长度应为 100+100-1=199。
y1 = zzh_lineconv(x(1,:),x(2,:));
figure;
plot(y1);
title('线性褶积结果');
xlabel('输出样点');
ylabel('幅值');
grid on;

% 2. 110 点循环褶积：两个输入先补零或截取到指定循环长度。
y2 = zzh_circonv(x(3,:),x(4,:),110);
figure;
plot(y2);
title('循环褶积结果');
xlabel('循环移位量');
ylabel('幅值');
grid on;

% 3. 线性相关：第二个序列平方后与第一个序列进行滑动相似性度量。
y3 = zzh_linecorr(x(5,:),x(6,:).^2);
figure;
```

```
plot(y3);
title('线性相关结果');
xlabel('相对位移');
ylabel('相关值');
grid on;

% 4. 110 点循环相关：序列按长度 110 周期延拓后计算相关。
y4 = zzh_circorr(x(7,:).^2,x(8,:),110);
figure;
plot(y4);
title('循环相关结果');
xlabel('循环位移量');
ylabel('相关值');
grid on;
```

Q2_02.m

```
% Q2_02.m
% 功能：数值验证两个重要性质：
% 1) 当 N>=N1+N2-1 时，补零后的 N 点循环褶积等于线性褶积；
% 2) 对实序列，线性相关等于“第二序列反序后”与第一序列的线性褶积。

clear;
clc;
close all;

x1 = [1,2,3,2];
x2 = [2,-1,1];
N = length(x1)+length(x2)-1;

lc = zzh_lineconv(x1,x2);
cc = zzh_circonv(x1,x2,N);

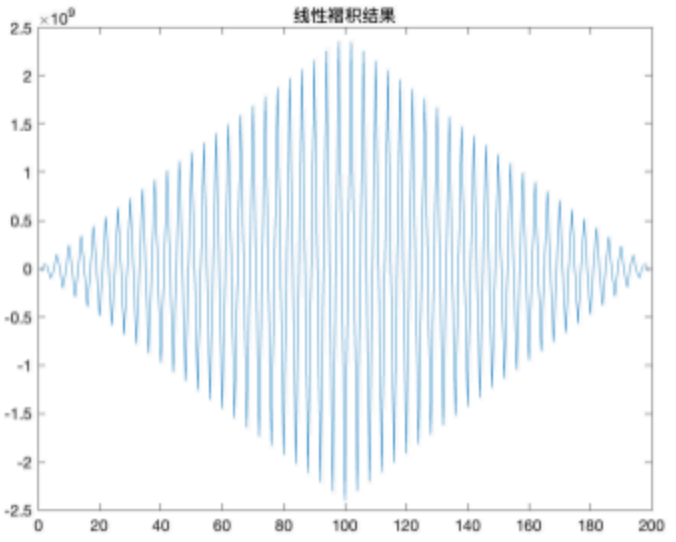
x2r = zeros(1,length(x2));
for i = 1:length(x2)
    x2r(i) = x2(length(x2)-i+1);
end
lr = zzh_linecorr(x1,x2);
rc = zzh_lineconv(x1,x2r);

figure;
plot(lc,'o-');
hold on;
plot(cc,'x--');
title('线性褶积与循环褶积对比');
```

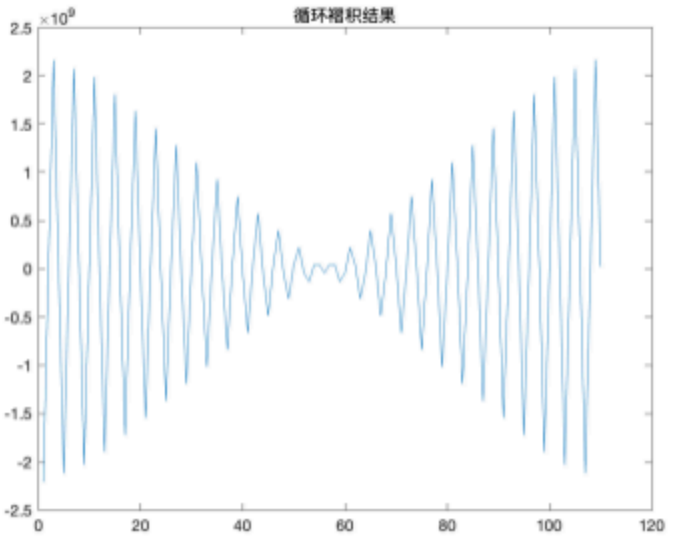
```
legend('线性褶积','循环褶积');
grid on;


figure;
plot(lr,'o–');
hold on;
plot(rc,'x––');
title('线性相关与线性褶积对比');
legend('线性相关','反序后线性褶积');
grid on;
```

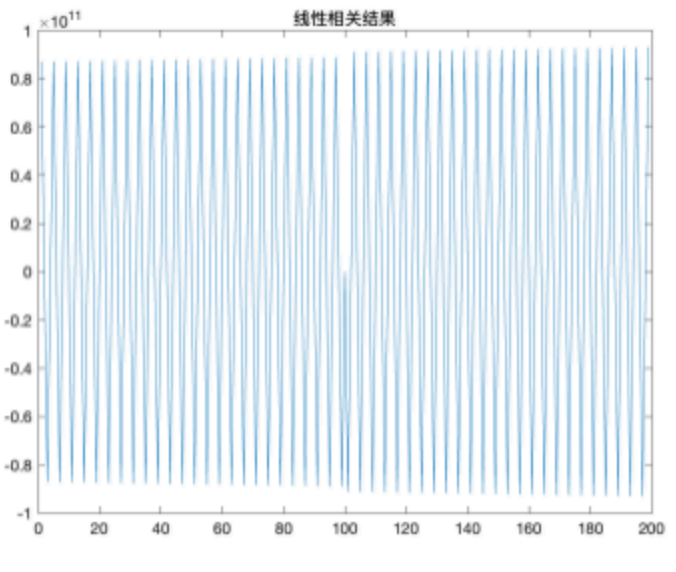
【运行结果】



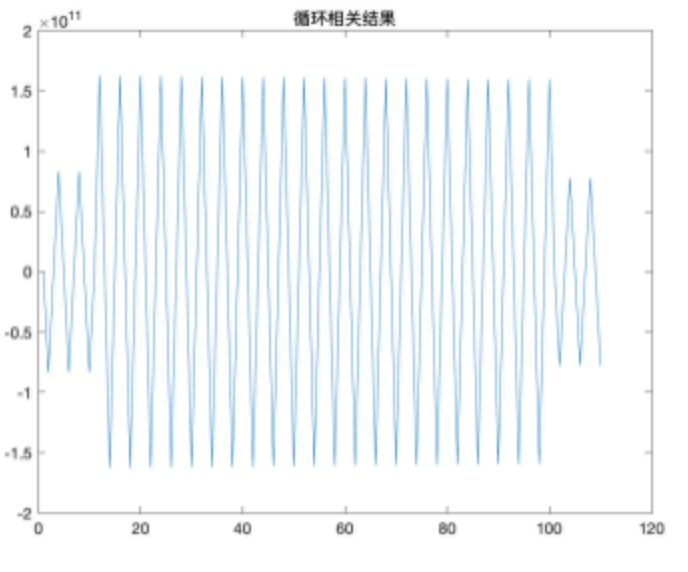
线性褶积



循环褶积



线性相关

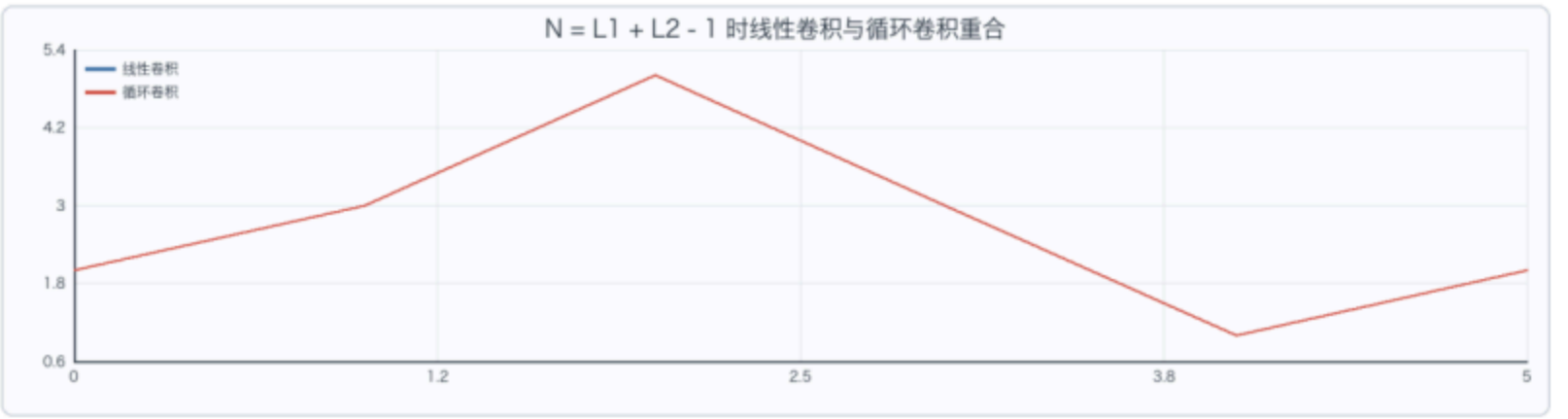


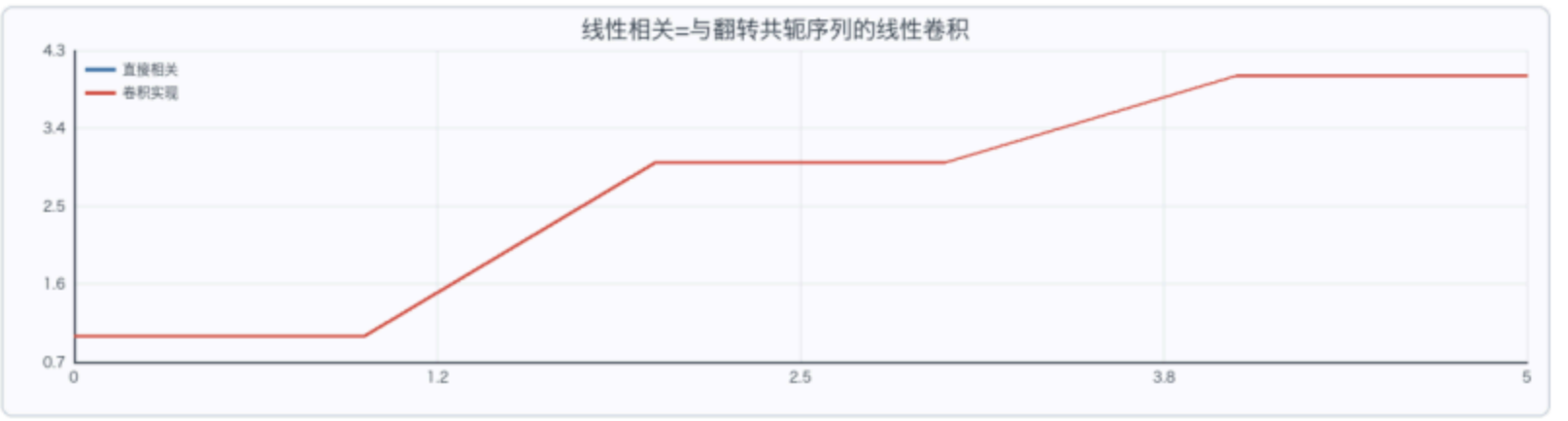
循环相关

【性质分析】

设两个序列长度分别为 L1 和 L2。当循环褶积长度 N≥L1+L2-1 时，补零后不会出现时域混叠，循环褶积的前 L1+L2-1 个有效值与线性褶积相同。线性相关的实现关系为先将 x2 逐点倒序，再与 x1 进行线性褶积。图中两组曲线重合，说明两条性质与程序实现一致。

卷积与相关的性质验证





循环褶积长度条件及相关与褶积关系验证

三、 DFT、IDFT 的 MATLAB 程序实现

用 MATLAB 程序分别实现 DFT 和 IDFT（要求写成 function 形式）．命名规则如下．（每小题 3 分．共 6 分）

【原理与编程思路】

N 点 DFT 的定义为 X[k]=Σx[n]exp(-j2πkn/N)，它把时域序列投影到 N 个离散复指数基函数上；|X[k]| 表示第 k 个离散频率处的幅值。IDFT 定义为 x[n]=(1/N)ΣX[k]exp(j2πkn/N)，与 DFT 相比指数符号相反，并多出归一化系数 1/N，因此 DFT 与 IDFT 构成可逆变换。程序严格按照定义使用两层 for 循环：DFT 外层遍历频率序号 k，内层遍历时域样点 n；IDFT 的循环次序相反但本质相同。数学下标从 0 开始，而 MATLAB 数组下标从 1 开始，所以访问数组时使用 n+1、k+1。测试信号含 3/32 和 7/32 两个频率分量，频谱应在对应的正负频率位置出现峰值；IDFT 重建曲线应与原信号重合。

【编程代码及详细注释：Q3_01.m】

```
function Xk = zzh_DFT(xn)
% 按定义式计算 N 点离散傅里叶变换。
% 输入：xn 为长度 N 的时域序列。
% 输出：Xk 为 N x 1 复数频谱列向量。
% 定义：X(k)=sum_{n=0}^{N-1} x(n)*exp(-j*2*pi*k*n/N)。
% 计算复杂度为 O(N^2)，用于展示 DFT 原理，不调用 fft。

N = length(xn);
xn = xn(:);
Xk = zeros(N,1);

for k = 0:N-1
    s = 0;
    for n = 0:N-1
        w = exp(-1j*2*pi*k*n/N);
        s = s + xn(n+1)*w;
    end
    Xk(k+1) = s;
end
end

function xn = zzh_IDFT(Xk)
% 按定义式计算 N 点离散傅里叶逆变换。
% 定义：x(n)=(1/N)*sum_{k=0}^{N-1} X(k)*exp(j*2*pi*k*n/N)。
```

```
% 与 DFT 相比，指数符号变为正，并在累加结束后除以 N。


N = length(Xk);
Xk = Xk(:);
xn = zeros(N,1);


for n = 0:N-1
    s = 0;
    for k = 0:N-1
        w = exp(1j*2*pi*k*n/N);
        s = s + Xk(k+1)*w;
    end


    % 理论上的实信号重建结果为实数；real 去除浮点运算产生的微小虚部。
    xn(n+1) = real(s)/N;
end
end
```

```
% Q3_01.m
% 功能：调用自编 DFT/IDFT，观察双频信号的频谱并验证可逆性。
% 信号包含归一化频率为 3/32 和 7/32 的两个正弦分量。


clear;
clc;
close all;


N = 32;
n = 0:N-1;
x = sin(2*pi*3*n/N) + 0.5*sin(2*pi*7*n/N);


X = zzh_DFT(x);
y = zzh_IDFT(X);


figure;
stem(0:N-1,abs(X));
title('DFT 振幅谱');
xlabel('频率序号 k');
ylabel('|X(k)|');
grid on;


figure;
plot(n,x,'o-');
hold on;
plot(n,y,'x--');
title('IDFT 重建结果');
```

```
xlabel('n');
ylabel('幅值');
legend('原信号','重建信号');
grid on;
```

【运行结果】



手写DFT 频谱及 IDFT 重建结果

DFT 后再进行 IDFT，重建波形与原信号重合，说明自编 DFT 和 IDFT 的变换方向、归一化系数以及下标换算是正确的。

四、 一维频域滤波器的 MATLAB 程序实现

调用上述编写的 DFT 与 IDFT 子函数，分别实现一维高通滤波器、一维低通滤波器、一维带通滤波器、一维带阻滤波器（要求写成 function 形式）。注意对滤波器进行镶边处理（梯形窗）。命名规则如下。（每小题 3 分。共 12 分）

【原理与编程思路】

频域滤波依据卷积定理实现: 时域卷积等价于频域相乘。程序先用自编 DFT 得到 X[k], 再构造滤波器频率响应 H[k]，令 Y[k]=X[k]H[k]，最后通过 IDFT 返回时域。DFT 的频率间隔为 Δf=fs/N；对实信号，频谱后半段对应负频率，因此使用 d=min(f,fs-f) 把正、负频率统一映射为距 0 Hz 的频率绝对值，从而构造对称响应。

低通窗在低频为 1、高频为 0；高通相反；带通仅在 f1～f2 为 1；带阻在 f1～f2 为 0。若使用理想矩形窗，截止处的突变会在时域产生较明显振铃，因此本设计在边缘加入线性上升或下降的过渡带，形成梯形窗。测试信号含 10、60、200 Hz 三个分量，可根据各滤波器的通阻带预测保留结果。

【编程代码及详细注释】

zzh_1DHP.m

```
function y = zzh_1DHP(x,fs,fc)
% 一维梯形窗高通频域滤波器。
% 低于 fc 的频率被抑制，高于 fc 的频率被保留；
% 截止频率下方设置线性过渡带，以减小矩形窗突变造成的时域振铃。


N = length(x);
H = zeros(N,1);
df = fs/N;
tr = max(df,0.2*fc);

for i = 1:N
    f = (i-1)*df;
    d = min(f,fs-f);

    if d >= fc
        H(i) = 1;
    elseif d > fc-tr
        H(i) = (d-fc+tr)/tr;
```

```
    end
end

X = zzh_DFT(x);
Y = X.*H;
y = zzh_IDFT(Y);

if size(x,1)==1
    y = y.';
end
end
```

zzh_1DLP.m

```
function y = zzh_1DLP(x,fs,fc)
% 一维梯形窗低通频域滤波器。
% 输入：x 为时域信号，fs 为采样频率，fc 为截止频率。
% 输出：y 为滤波后的时域信号。
% 方法：构造关于 0 Hz 对称的频率窗 H，完成 DFT->乘窗->IDFT。

N = length(x);
H = zeros(N,1);
df = fs/N;
tr = max(df,0.2*fc);

for i = 1:N
    f = (i-1)*df;
    d = min(f,fs-f);

    if d <= fc
        H(i) = 1;
    elseif d < fc+tr
        H(i) = (fc+tr-d)/tr;
    end
end

X = zzh_DFT(x);
Y = X.*H;
y = zzh_IDFT(Y);

% 保持输出方向与输入一致，方便后续程序直接相减或绘图。
if size(x,1)==1
    y = y.';
end
end
```

zzh_1DBP.m

```
function y = zzh_1DBP(x,fs,f1,f2)
% 一维梯形窗带通频域滤波器。
% f1~f2 为通带；两侧设置线性过渡区，区外频率增益为 0。
% d=min(f,fs-f) 用于把 DFT 后半段映射为对应的负频率绝对值，
% 从而保证实信号滤波器具有共轭对称的频率响应。

N = length(x);
H = zeros(N,1);
df = fs/N;
tr = max(df,0.1*(f2-f1));

for i = 1:N
    f = (i-1)*df;
    d = min(f,fs-f);

    if d >= f1 && d <= f2
        H(i) = 1;
    elseif d > f1-tr && d < f1
        H(i) = (d-f1+tr)/tr;
    elseif d > f2 && d < f2+tr
        H(i) = (f2+tr-d)/tr;
    end
end

X = zzh_DFT(x);
Y = X.*H;
y = zzh_IDFT(Y);

if size(x,1)==1
    y = y.';
end
end
```

zzh_1DBE.m

```
function y = zzh_1DBE(x,fs,f1,f2)
% 一维梯形窗带阻频域滤波器。
% f1~f2 为阻带，阻带外增益为 1；两个边界用线性镶边形成梯形窗。
% 适合压制频率位置已知的窄带干扰，例如 50 Hz 工业电干扰。

N = length(x);
H = ones(N,1);
df = fs/N;
tr = max(df,0.1*(f2-f1));
```

```
for i = 1:N
    f = (i-1)*df;
    d = min(f,fs-f);

    if d >= f1 && d <= f2
        H(i) = 0;
    elseif d > f1-tr && d < f1
        H(i) = (f1-d)/tr;
    elseif d > f2 && d < f2+tr
        H(i) = (d-f2)/tr;
    end
end

X = zzh_DFT(x);
Y = X.*H;
y = zzh_IDFT(Y);

if size(x,1)==1
    y = y.';
end
end
```

Q4_01.m

```
% Q4_01.m
% 功能：构造含 10 Hz、60 Hz、200 Hz 三个分量的信号，测试四类频域滤波器。
% 方法：DFT -> 与梯形频率窗逐点相乘 -> IDFT。

clear;
clc;
close all;

fs = 1000;
T = 0.5;
t = 0:1/fs:T-1/fs;

% 三个频率分量的幅值分别为 1、0.7、0.5。
x = sin(2*pi*10*t) + 0.7*sin(2*pi*60*t) + 0.5*sin(2*pi*200*t);

lp = zzh_1DLP(x,fs,20);
hp = zzh_1DHP(x,fs,100);
bp = zzh_1DBP(x,fs,40,80);
be = zzh_1DBE(x,fs,40,80);
```

```
figure;
plot(t,x);
title('原始信号');
xlabel('时间/s');
grid on;

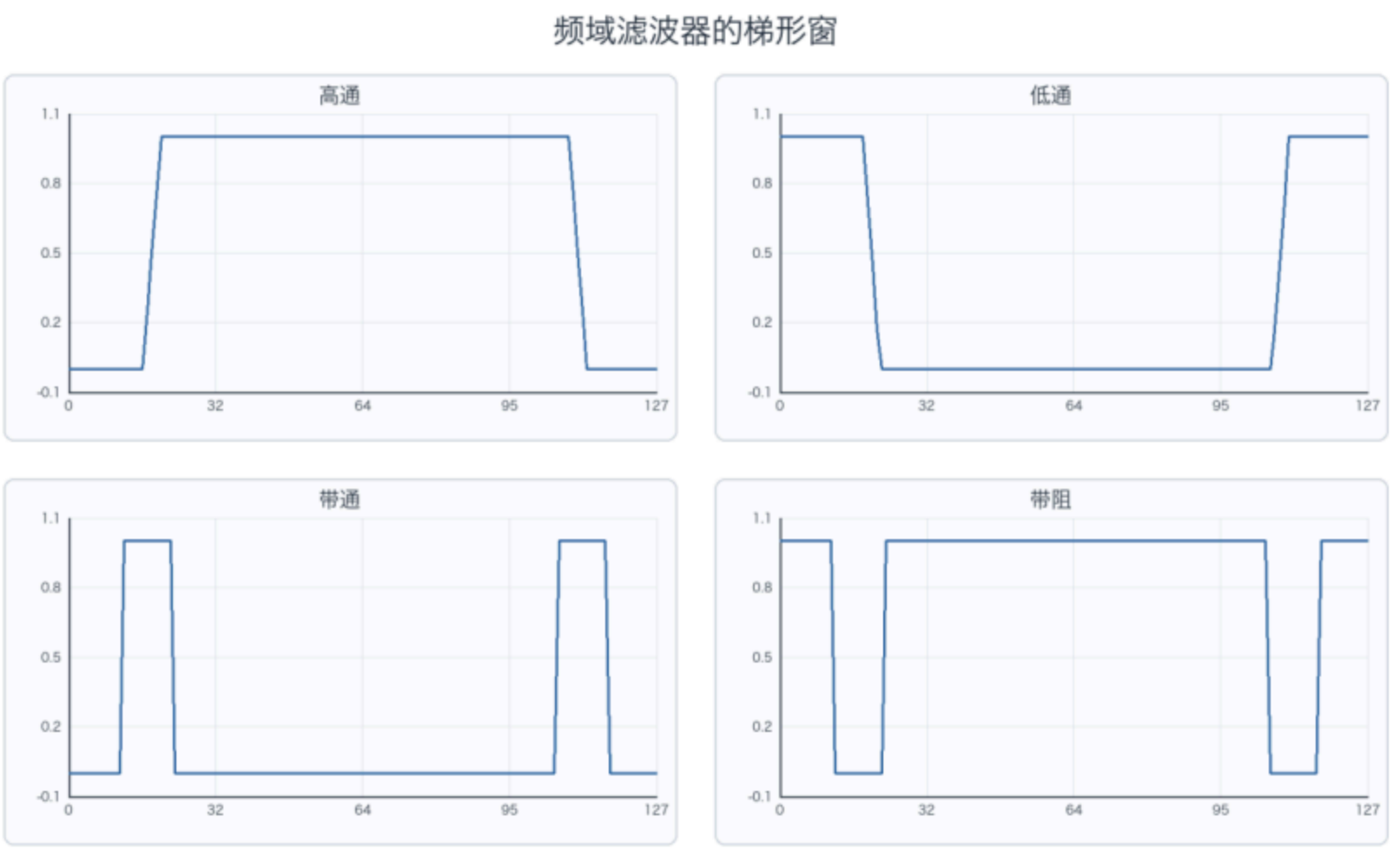
figure;
plot(t,lp);
title('低通滤波：保留低频分量');
xlabel('时间/s');
grid on;

figure;
plot(t,hp);
title('高通滤波：保留高频分量');
xlabel('时间/s');
grid on;

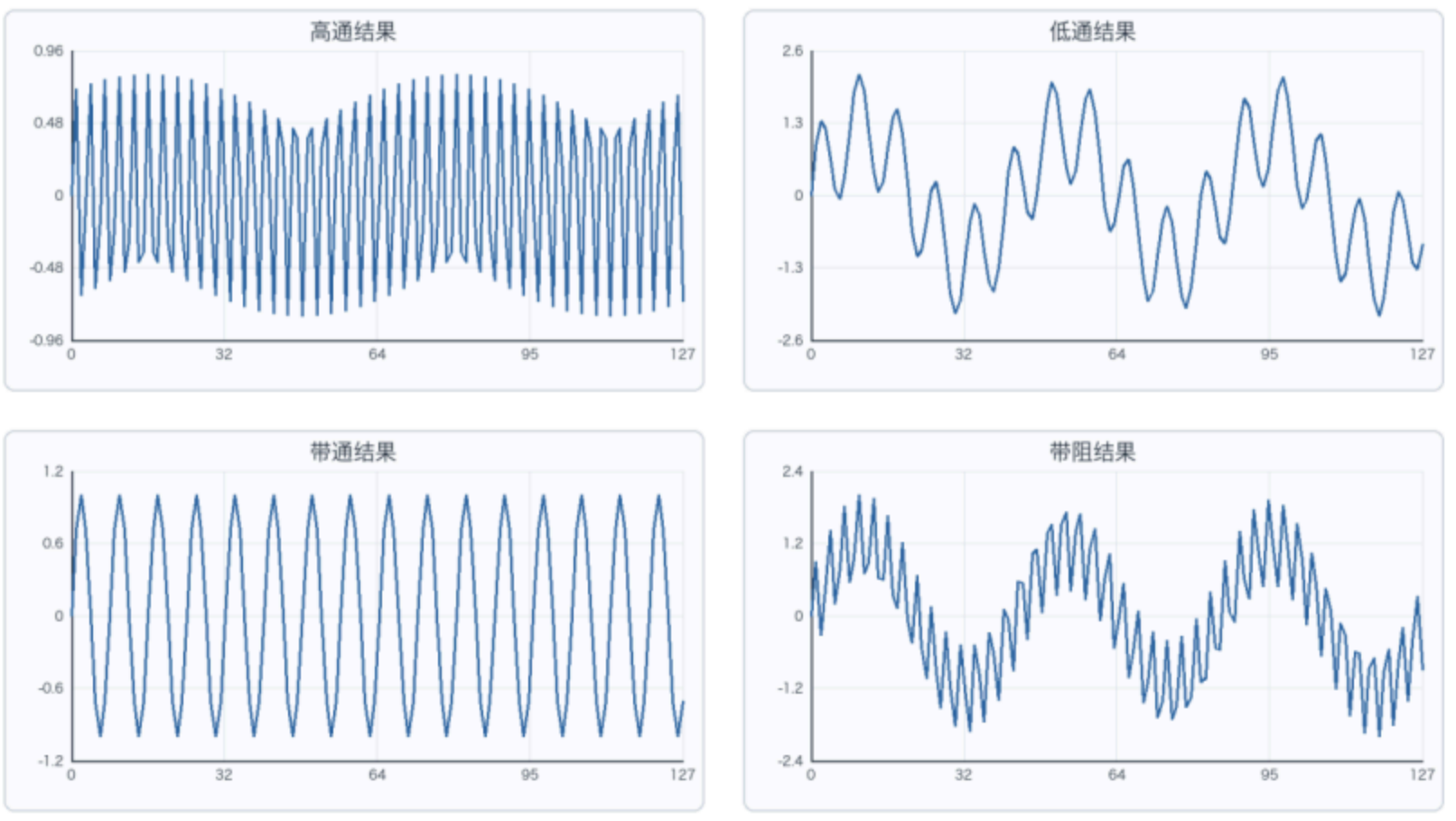
figure;
plot(t,bp);
title('带通滤波：保留中间频带');
xlabel('时间/s');
grid on;

figure;
plot(t,be);
title('带阻滤波：抑制中间频带');
xlabel('时间/s');
grid on;
```

【运行结果】



四种频域滤波器的梯形窗



四种频域滤波器的处理结果

结果表明，不同滤波器分别保留或抑制了指定频段，梯形过渡区使频率响应边缘更加平缓。

五、 一维时域滤波器的 MATLAB 程序实现

用 MATLAB 程序分别实现均值滤波、三角滤波、高斯滤波、中值滤波．命名规则如下．(每小题 3 分．共 12 分)

【原理与编程思路】

时域平滑滤波直接利用当前样点附近的局部信息。均值滤波给窗口内各点相同权值，能够降低零均值随机噪声，但会模糊突变；三角滤波使中心权值最大、向两侧线性减小，比等权平均更重视局部中心；高斯滤波采用 $\exp(-j^2/(2\sigma^2))$ 权值，随距离平滑衰减；中值滤波先排序再取中间值，属于非线性滤波，对孤立脉冲和野值最稳健。

四个函数都先把输入统一为行向量，并为输出预分配空间。对每个位置建立局部窗口，边界处只使用落在有效下标范围内的样点。加权滤波必须用实际有效权值和归一化，避免边界幅值被压低；中值滤波在有效点数为偶数时取中间两点平均。Q5_01.m 固定随机种子，使不同次运行能够得到完全相同的对比结果。

【编程代码及详细注释】

zzh_1DMF.m

```
function y = zzh_1DMF(x,M)
% 一维移动均值滤波器 (Mean Filter)。
% 输入：x 为待滤波序列，M 为窗口长度。
% 每个输出点等于其邻域内有效样点的算术平均值。
% 边界处只统计落在信号范围内的样点，因此不会人为补零降低幅值。


x = x(:).';
N = length(x);
r = floor(M/2);
y = zeros(1,N);

for i = 1:N
    s = 0;
    c = 0;
    for j = i-r:i+r
        if j >= 1 && j <= N
            s = s + x(j);
            c = c + 1;
        end
```

```
    end
    y(i) = s/c;
end
end
```

zzh_1DTF.m

```
function y = zzh_1DTF(x,M)
% 一维三角加权滤波器（Triangle Filter）。
% 权值在窗口中心最大，向两侧按距离线性减小：
% w(j)=r+1−|j|。与均值滤波相比，它更重视中心附近样点。

x = x(:).';
N = length(x);
r = floor(M/2);
y = zeros(1,N);

for i = 1:N
    s = 0;
    sw = 0;
    for j = −r:r
        p = i+j;
        if p >= 1 && p <= N
            w = r+1−abs(j);
            s = s + w*x(p);
            sw = sw + w;
        end
    end
    y(i) = s/sw;
end
end
```

zzh_1DGF.m

```
function y = zzh_1DGF(x,sg)
% 一维高斯加权滤波器（Gaussian Filter）。
% sg 控制高斯权值的离散程度；窗口截取为 +/−3*sg，
% 因为该范围已覆盖高斯函数绝大部分能量。

x = x(:).';
N = length(x);
r = ceil(3*sg);
y = zeros(1,N);

for i = 1:N
    s = 0;
```

```
    sw = 0;
    for j = -r:r
        p = i+j;
        if p >= 1 && p <= N
            w = exp(-(j*j)/(2*sg*sg));
            s = s + w*x(p);
            sw = sw + w;
        end
    end
    y(i) = s/sw;
end
end
```

zzh_1DMidF.m

```
function y = zzh_1DMidF(x,M)
% 一维中值滤波器（Median Filter）。
% 对每个窗口中的样点排序并取中间值，能够有效抑制孤立尖峰和野值。
% 若边界处有效样点数为偶数，则取中间两个值的平均。

x = x(:).';
N = length(x);
r = floor(M/2);
y = zeros(1,N);

for i = 1:N
    data = zeros(1,M);
    c = 0;
    for j = i-r:i+r
        if j >= 1 && j <= N
            c = c+1;
            data(c) = x(j);
        end
    end
    data = sort(data(1:c));

    h = floor(c/2);
    if c == 2*h
        % 偶数个样点没有唯一中心值，取中间两点平均。
        y(i) = (data(h)+data(h+1))/2;
    else
        % 奇数个样点直接取排序后的正中间元素。
        y(i) = data(h+1);
    end
end
```

```
end
```

Q5_01.m

```
% Q5_01.m
% 功能：比较均值、三角、高斯和中值四种一维时域滤波方法。
% 对比重点：前三种属于加权平滑，中值滤波属于非线性排序统计滤波。

close all;
clear;
clc;

N = 200;
n = 0:N-1;
x0 = sin(2*pi*0.05*n) + 0.4*sin(2*pi*0.13*n);

rng(2);
xn = x0 + 0.6*randn(1,N);

ym = zzh_1DMF(xn,5);
yt = zzh_1DTF(xn,5);
yg = zzh_1DGF(xn,1.0);
yd = zzh_1DMidF(xn,5);

figure('Name','无噪信号');
plot(x0);
grid on;
title('无噪信号');
print(gcf,'results/09_时域滤波_1_无噪信号.png','-dpng','-r150');

figure('Name','含噪信号');
plot(xn);
grid on;
title('含噪信号');
print(gcf,'results/09_时域滤波_2_含噪信号.png','-dpng','-r150');

figure('Name','均值滤波');
plot(ym);
grid on;
title('均值滤波');
print(gcf,'results/09_时域滤波_3_均值滤波.png','-dpng','-r150');

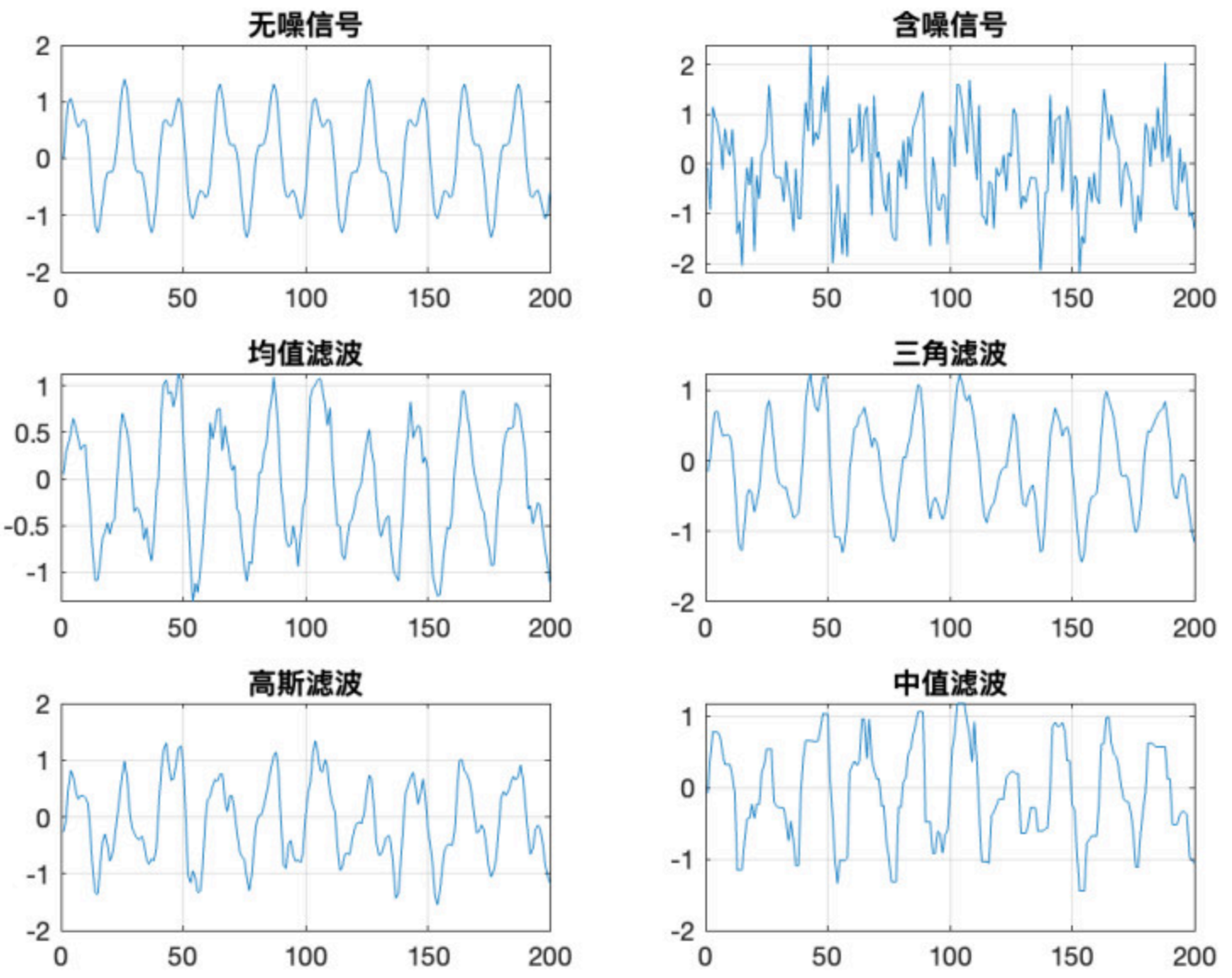
figure('Name','三角滤波');
plot(yt);
grid on;
```

```
title('三角滤波');
print(gcf,'results/09_时域滤波_4_三角滤波.png','-dpng','-r150');

figure('Name','高斯滤波');
plot(yg);
grid on;
title('高斯滤波');
print(gcf,'results/09_时域滤波_5_高斯滤波.png','-dpng','-r150');

figure('Name','中值滤波');
plot(yd);
grid on;
title('中值滤波');
print(gcf,'results/09_时域滤波_6_中值滤波.png','-dpng','-r150');
```

【运行结果】



四种一维时域滤波器处理结果

四种方法均减弱了快速随机波动，但特性不同：均值滤波平滑程度直接，容易同时削弱局部峰值；三角和高斯滤波对中心样点权重更高，波形过渡更自然；中值滤波并非线性平均，对孤立尖峰最不敏感。选择窗口越大，降噪通常越强，但有效信号细节损失也越明显，因此窗口长度需要在噪声抑制和波形保真之间折中。

六、 一维信号处理实战演练

自选调用、合并、或改写以上编写函数，进行如下一维信号的噪声压制处理（设法提高信噪比），并计算处理结果的信噪比（每小题 4 分，共 20 分）

要求：画出 1）处理后的时间域信号；2）处理后信号的振幅谱；3）处理后的时间域信号与无噪信号之差。并附上源代码及注释。

（信噪比公式：10*log10(Ps/Pn)，其中 Ps 为无噪数据的功率，Pn 为处理结果与无噪数据之差的功率。）

1) 压制低频噪声
2) 压制高频噪声
3) 压制随机噪声
4) 压制野值噪声
5) 压制工业电干扰

【实验信号与处理方法】

实验以 signal 作为无噪参考，n1～n5 分别对应低频、高频、随机、野值和工业电干扰。滤波方法应与噪声的频谱或统计特征匹配：低频噪声用高通去除，高频噪声用低通限制带宽；随机噪声分布较宽，先低通保留有效频带，再用移动平均减小局部随机波动；野值是少量幅值异常点，中值滤波能在不被极端值显著影响的情况下恢复局部趋势；50 Hz 工业电干扰频率集中，使用窄带带阻滤波最有针对性。

每项处理分别绘制处理后时域信号、单边振幅谱以及与 signal 的误差。评价指标采用 SNR=10log10(Ps/Pn)，其中 Ps 是参考信号平均功率，Pn 是处理结果与参考信号之差的平均功率。处理后 SNR 高于处理前说明误差功率下降，但还需结合波形和频谱判断是否误删有效信号。

【编程代码及详细注释：Q6_01.m】

```
% Q6_01.m
% 功能：针对五类一维噪声选择相应滤波方法，并以信噪比定量评价效果。
% 数据说明：signal 为无噪参考信号；n1~n5 依次为五种含噪信号。

close all;
clear;
clc;
load data_1D.mat;

fs = 1000;
N = length(signal);
t = (0:N-1)/fs;

% 1. 低频噪声：噪声集中在信号有效频带下方，使用高通滤波器抑制。
```

```
y1 = zzh_1DHP(n1,fs,25);
showR(t,signal,n1,y1,fs, ...
                '低频噪声','results/10_低频噪声.png');

% 2. 高频噪声：有效信号主要位于低频，使用低通滤波器截除高频成分。
y2 = zzh_1DLP(n2,fs,55);
showR(t,signal,n2,y2,fs, ...
                '高频噪声','results/11_高频噪声.png');

% 3. 随机噪声：先限制频带，再用 7 点均值滤波进一步平滑随机波动。
tmp = zzh_1DLP(n3,fs,55);
y3 = zzh_1DMF(tmp,7).';
showR(t,signal,n3,y3,fs, ...
                '随机噪声','results/12_随机噪声.png');

% 4. 野值噪声：孤立尖峰会显著偏离邻域，中值滤波对此类噪声较稳健。
y4 = zzh_1DMidF(n4,5).';
showR(t,signal,n4,y4,fs, ...
                '野值噪声','results/13_野值噪声.png');

% 5. 工业电干扰：50 Hz 附近为窄带周期干扰，采用窄带带阻滤波器。
y5 = zzh_1DBE(n5,fs,50,50.5);
showR(t,signal,n5,y5,fs, ...
                '工业电干扰','results/14_工业电干扰.png');

% 输出处理前后的 SNR，处理后数值越大表示误差功率越小。
fprintf('\n 一维信号信噪比汇总（dB）\n');
fprintf('低频噪声：处理前 %.2f，处理后 %.2f\n', ...
            zzh_snr(signal,n1),zzh_snr(signal,y1));
fprintf('高频噪声：处理前 %.2f，处理后 %.2f\n', ...
            zzh_snr(signal,n2),zzh_snr(signal,y2));
fprintf('随机噪声：处理前 %.2f，处理后 %.2f\n', ...
            zzh_snr(signal,n3),zzh_snr(signal,y3));
fprintf('野值噪声：处理前 %.2f，处理后 %.2f\n', ...
            zzh_snr(signal,n4),zzh_snr(signal,y4));
fprintf('工业电干扰：处理前 %.2f，处理后 %.2f\n', ...
            zzh_snr(signal,n5),zzh_snr(signal,y5));

function showR(t,x0,xn,y,fs,name,pic)
% 统一绘制处理结果、振幅谱和相对参考信号的误差。

    [f,A] = zzh_ampspec(y,fs);
    [p,b,e] = fileparts(pic);
```

```
    figure('Name',[name,'时域']);
    plot(t,y);
    grid on;
    title([name,'压制后的时间域信号']);
    xlabel('时间/s');
    ylabel('幅值');
    print(gcf,fullfile(p,[b,'_1_时域',e]),'-dpng','-r150');

    figure('Name',[name,'频谱']);
    plot(f,A);
    grid on;
    title('处理后信号的振幅谱');
    xlabel('频率/Hz');
    ylabel('振幅');
    print(gcf,fullfile(p,[b,'_2_频谱',e]),'-dpng','-r150');

    figure('Name',[name,'误差']);
    err = y-x0;
    plot(t,err);
    grid on;
    title(['处理误差，处理前SNR=',num2str(zzh_snr(x0,xn),'%.2f'), ...
           ' dB，处理后SNR=',num2str(zzh_snr(x0,y),'%.2f'),' dB']);
    xlabel('时间/s');
    ylabel('误差');
    print(gcf,fullfile(p,[b,'_3_误差',e]),'-dpng','-r150');
end
```

信噪比计算函数

```
function v = zzh_snr(x0,y)
% 按课程给定公式计算信噪比（单位 dB）。
% x0 为无噪参考信号，y 为含噪或处理后的信号。
% 信号功率 Ps=mean(|x0|^2);
% 误差功率 Pn=mean(|y-x0|^2);
% SNR=10*log10(Ps/Pn)。

x0 = x0(:);
y = y(:);

Ps = 0;
Pn = 0;
for k = 1:length(x0)
    Ps = Ps + abs(x0(k))^2;
    Pn = Pn + abs(y(k)-x0(k))^2;
end
```

```
Ps = Ps/length(x0);
Pn = Pn/length(x0);

if Pn == 0
    v = inf;
else
    v = 10*log10(Ps/Pn);

    % 避免把数值舍入误差显示成极小的非零 dB。
    if abs(v) < 0.005
        v = 0;
    end
end
end
```

振幅谱计算函数

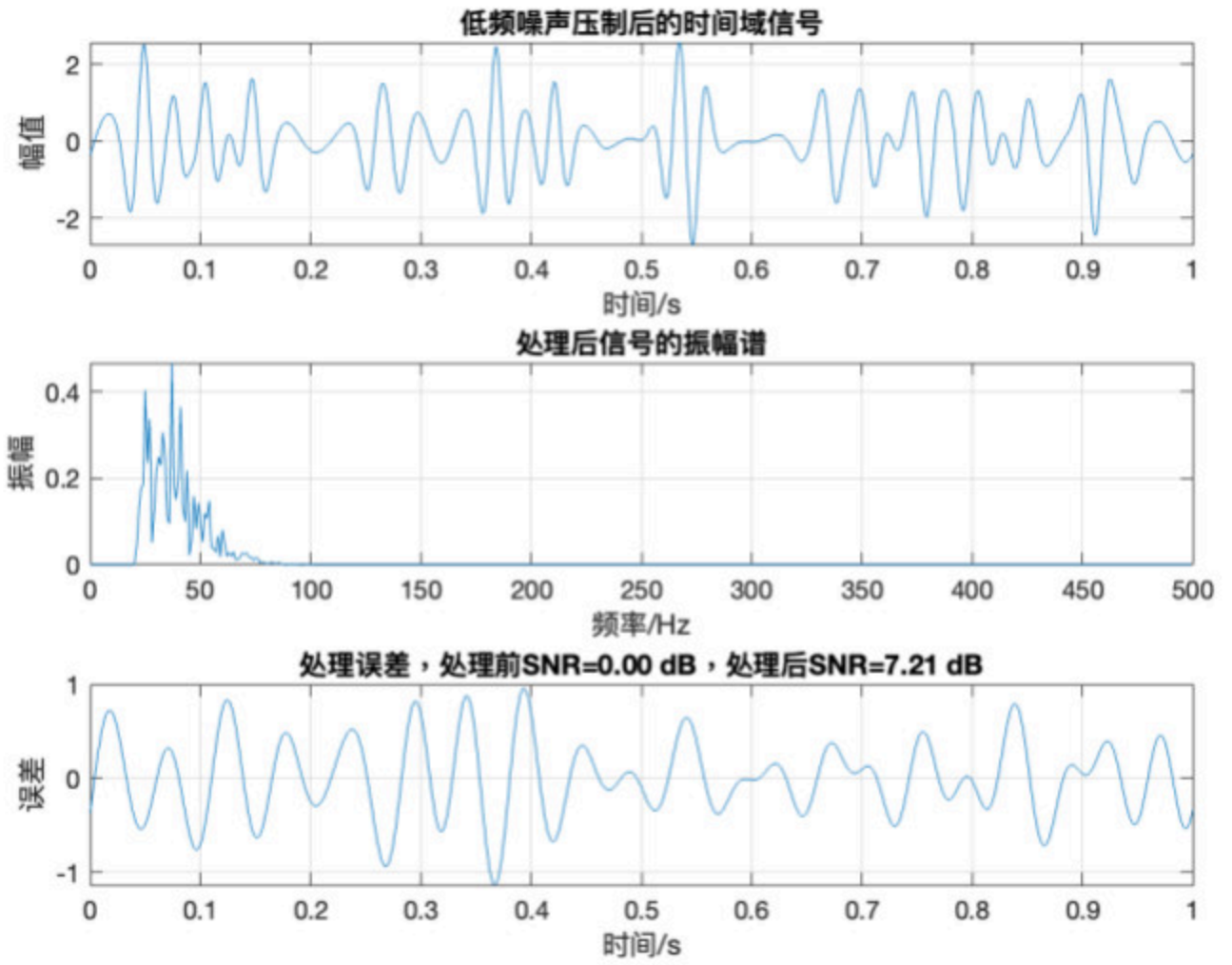
```
function [f,A] = zzh_ampspec(x,fs)
% 计算实信号的单边振幅谱。
% 输入：x 为时域信号，fs 为采样频率。
% 输出：f 为 0~Nyquist 附近的频率坐标，A 为对应单边振幅。

N = length(x);
X = zzh_DFT(x);
h = floor(N/2)+1;
f = (0:h-1)*fs/N;
A = zeros(1,h);

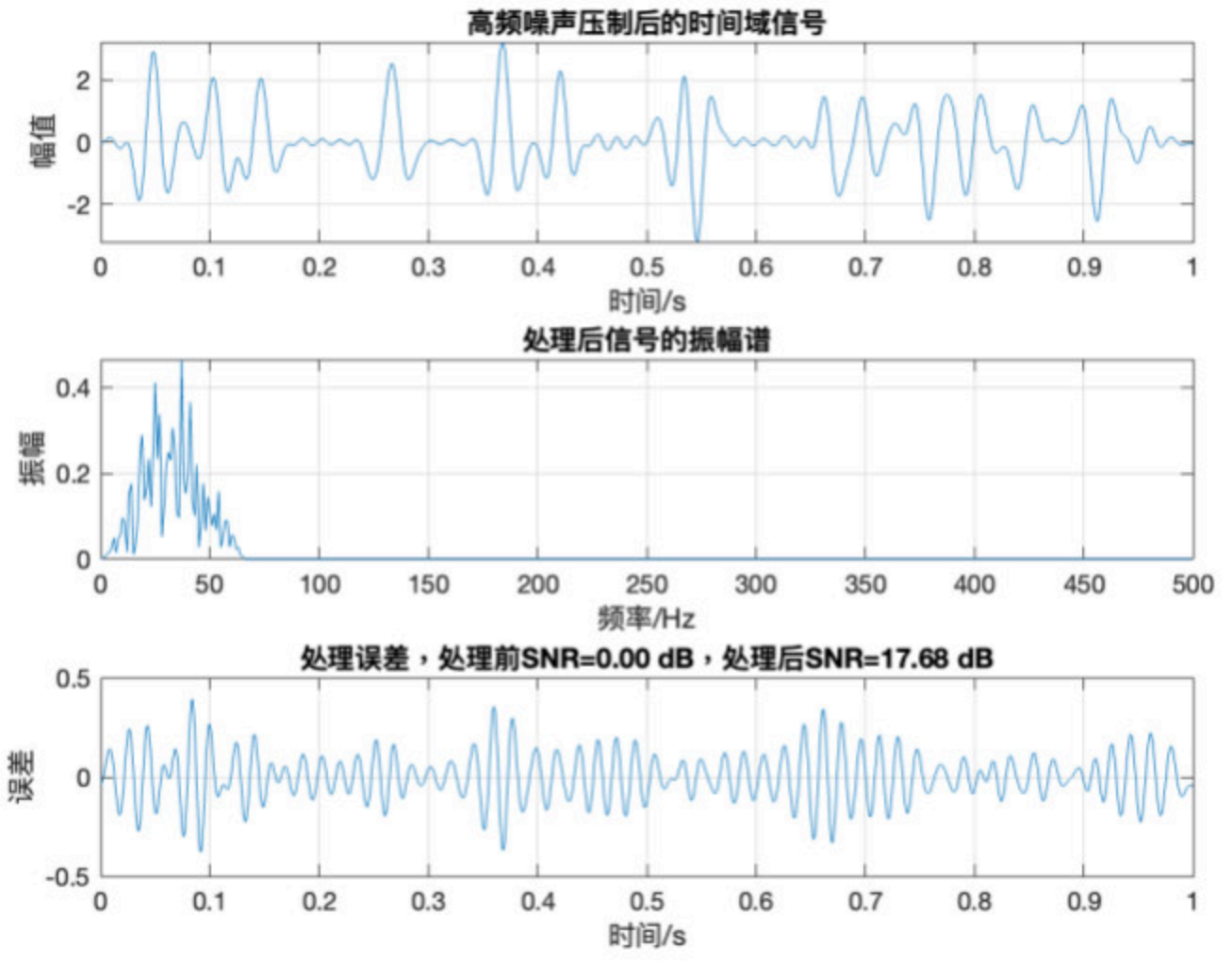
% 单边谱把负频率侧的幅值合并到正频率侧，因此一般乘以 2/N。
for k = 1:h
    A(k) = 2*abs(X(k))/N;
end

A(1) = A(1)/2;
end
```

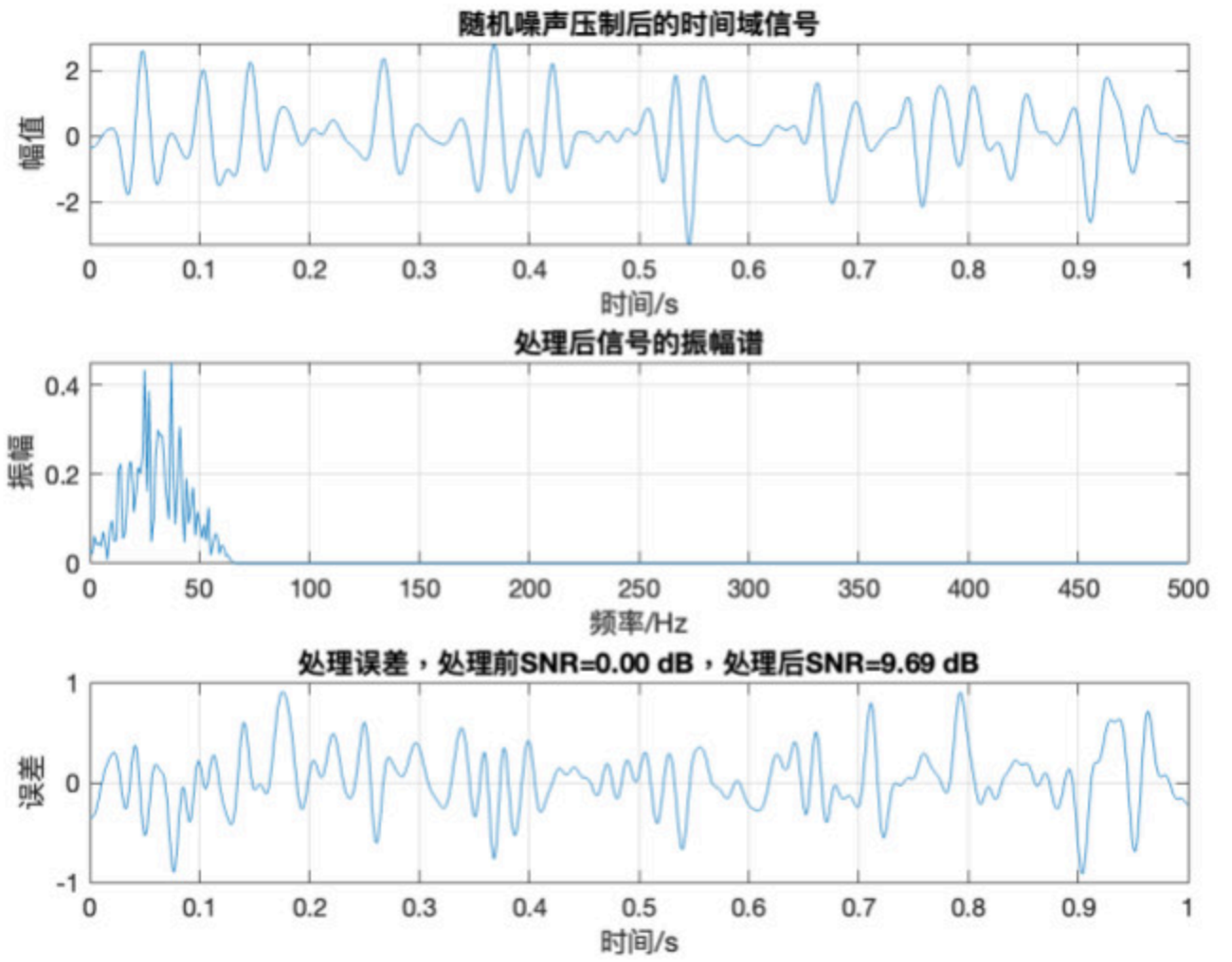
【运行结果与分析】



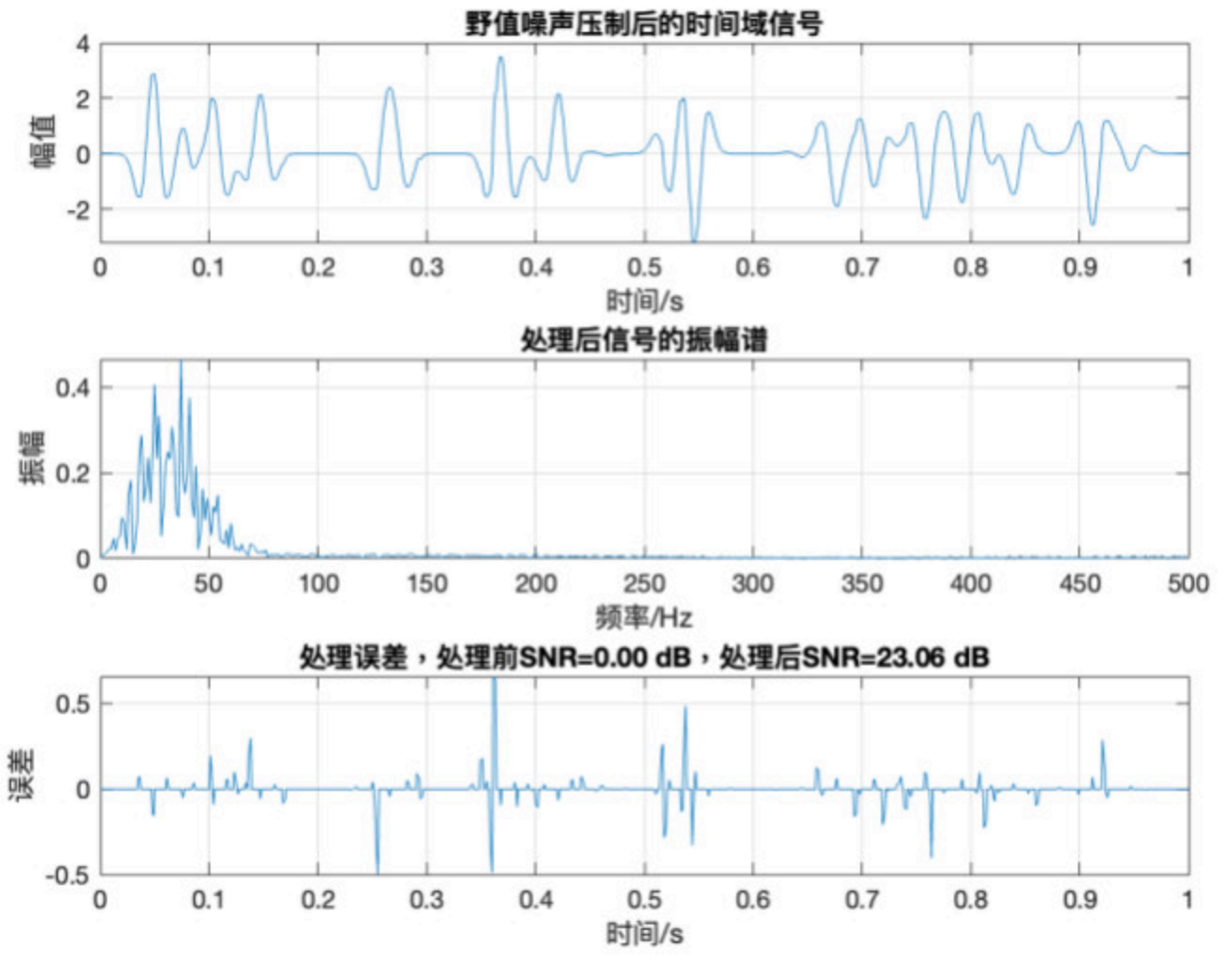
低频噪声压制结果



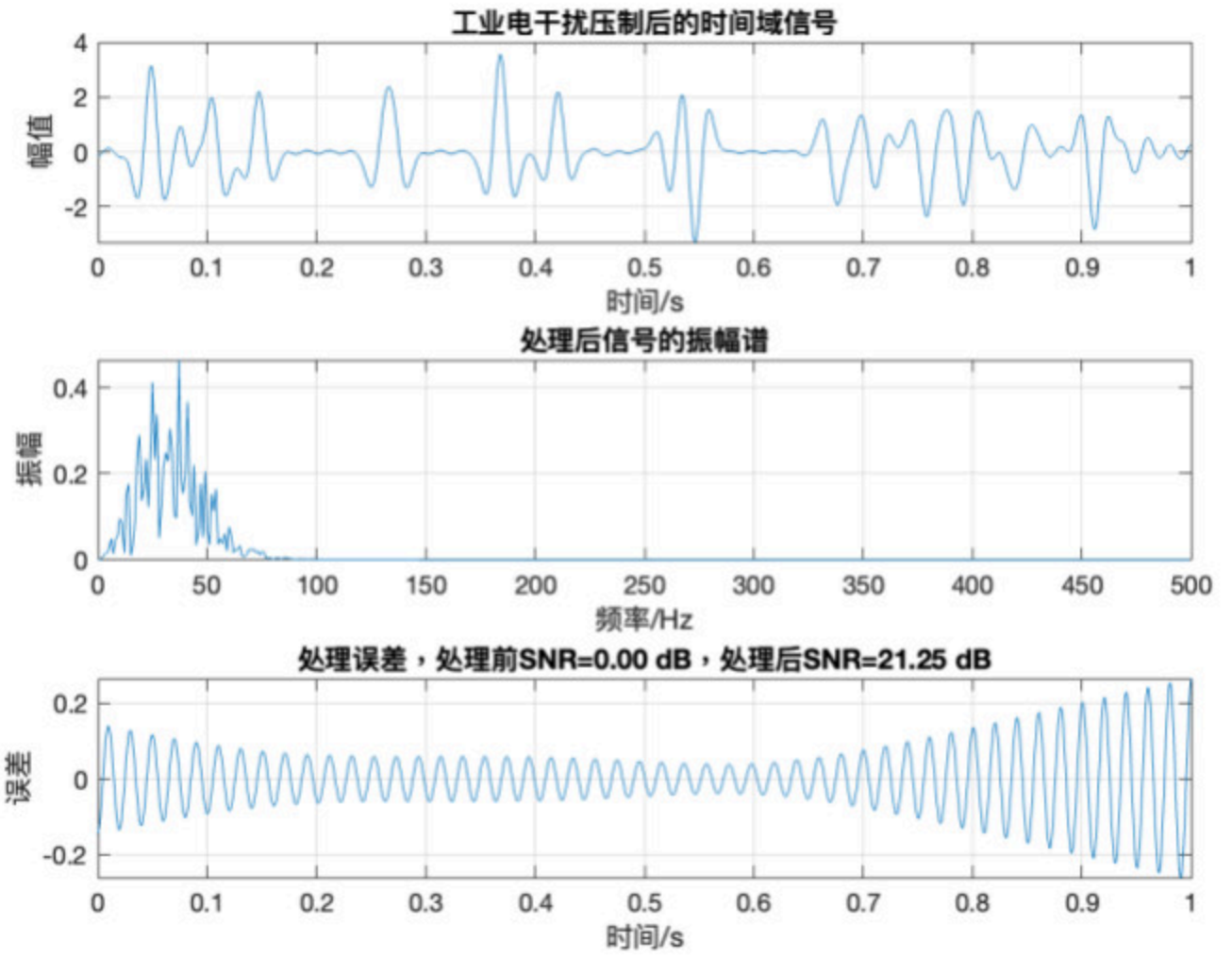
高频噪声压制结果



随机噪声压制结果



野值噪声压制结果



工业电干扰压制结果

五种处理结果的信噪比均高于处理前，说明相对于无噪参考信号的误差功率下降。低频、高频和工业电干扰具有较明确的频带位置，频域滤波能够有针对性地压制；随机噪声频带较宽，仅做低通仍可能残留，因此叠加均值平滑；野值噪声在时域中表现为孤立尖峰，中值滤波后异常峰值明显减少。结果同时表明：滤波器参数不能只追求更高 SNR，还应检查有效频率成分和波形细节是否被过度削弱。

七、 二维地震信号处理实战演练

自选调用、合并、或改写以上编写的函数，解决如下处理问题，并计算处理结果的信噪比。(每小题 4 分，共 12 分)

要求：画出 1) 处理后的时间域信号；2) 处理后信号的二维振幅谱 (FK 谱)；3) 处理后的时间域信号与无噪信号之差。并附上源代码及注释。

(信噪比公式：10*log10(Ps/Pn)，其中 Ps 为无噪数据的功率，Pn 为处理果与无噪数据之差的功率。)

1) 随机噪声压制
2) 面波压制
3) 缺道恢复

【实验数据与处理方法】

二维地震记录的两个维度分别是时间采样点和空间道号。随机噪声在相邻时间、相邻道之间相关性弱，而有效反射同相轴具有一定连续性，因此使用 3×3 二维均值可提高局部相干能量占比。面波通常具有低频、强能量特征；程序逐道用 21 点移动平均估计低频趋势，再从原道中相减，相当于简易高通处理。缺道表现为整列全零，先自动检测缺失道号，再使用左右最近有效道的等权平均进行空间插值；位于边界时复制唯一可用的邻道。

二维 DFT 利用可分离性实现：先对每列进行一维 DFT，再对中间结果的每行进行 DFT。显示 FK 谱前用二维循环移位把零频移到中心，并采用 log10(1+|F|) 压缩动态范围。每项实验分别比较处理剖面、FK 谱、误差剖面和 SNR，以避免只凭图像颜色判断处理效果。

【编程代码及详细注释】

Q7_01.m

```
% Q7_01.m
% 功能：对二维地震记录完成随机噪声压制、面波压制和缺道恢复。
% 数据说明：shotc 为无噪参考剖面；shot1~shot3 为三类待处理数据。

close all;
clear;
clc;
load data_2D.mat;

% 1. 随机噪声压制
% 在“时间采样点–道号”二维邻域内取 3x3 均值，利用随机噪声均值趋近于零的特点。
y1 = zzh_2Dmean(shot1,3,3);
showS(shotc,shot1,y1,'随机噪声压制', ...
                'results/15_二维随机噪声.png');
```

```
% 2. 面波压制
% 面波通常表现为强低频成分。逐道估计 21 点移动平均低频趋势，再从原道中相减。
[nt,nx] = size(shot2);
y2 = zeros(nt,nx);
for j = 1:nx
    lo = zzh_1DMF(shot2(:,j),21);
    y2(:,j) = shot2(:,j)-lo.';
end
showS(shotc,shot2,y2,'面波压制', ...
        'results/16_面波压制.png');

% 3. 缺道恢复
% 若某一整道的绝对值和为 0，则判定该道缺失；随后进行相邻有效道插值。
mis = [];
for j = 1:size(shot3,2)
    if sum(abs(shot3(:,j))) == 0
        mis = [mis,j];
    end
end
y3 = zzh_tracefill(shot3,mis);
showS(shotc,shot3,y3,'缺道恢复', ...
         'results/17_缺道恢复.png');

% 统一输出三项实验处理前后的信噪比。
fprintf('\n 二维地震信号信噪比汇总（dB）\n');
fprintf('随机噪声：处理前 %.2f，处理后 %.2f\n', ...
        zzh_snr(shotc,shot1),zzh_snr(shotc,y1));
fprintf('面波干扰：处理前 %.2f，处理后 %.2f\n', ...
        zzh_snr(shotc,shot2),zzh_snr(shotc,y2));
fprintf('缺道数据：处理前 %.2f，处理后 %.2f\n', ...
        zzh_snr(shotc,shot3),zzh_snr(shotc,y3));


function showS(x0,xn,y,name,pic)
% 显示处理剖面、FK 谱和相对参考剖面的误差。

    % 自编二维 DFT 计算量较大，因此抽取部分时间点和道后再计算 FK 谱。
    sm = y(1:10:end,1:2:end);
    F = zzh_DFT2(sm);

    % 把零频移到图像中心，并用对数压缩扩大弱频谱成分的可见范围。
    FK = log10(1 + abs(zzh_shift2(F)));
    e = y-x0;

    % 对称色标使正、负振幅具有一致的颜色动态范围。
```

```
c1 = max(abs(y(:)));
c2 = max(abs(e(:)));
[p,b,ex] = fileparts(pic);

figure('Name',[name,'剖面']);
imagesc(y);
axis tight;
colormap(jet(256));
caxis([-c1,c1]);
title([name,'后的时间域剖面']);
xlabel('道号');
ylabel('采样点');
print(gcf,fullfile(p,[b,'_1_剖面',ex]),'-dpng','-r150');

figure('Name',[name,'FK 谱']);
imagesc(FK);
axis tight;
colormap(hot(256));
title('处理后二维振幅谱（FK 谱）');
xlabel('空间频率');
ylabel('时间频率');
print(gcf,fullfile(p,[b,'_2_FK 谱',ex]),'-dpng','-r150');

figure('Name',[name,'误差']);
imagesc(e);
axis tight;
colormap(jet(256));
caxis([-c2,c2]);
title(['处理误差，SNR: ',num2str(zzh_snr(x0,xn),'%.2f'), ...
       '→',num2str(zzh_snr(x0,y),'%.2f'),' dB']);
xlabel('道号');
ylabel('采样点');
print(gcf,fullfile(p,[b,'_3_误差',ex]),'-dpng','-r150');
end
```

zzh_2Dmean.m

```
function y = zzh_2Dmean(x,mt,mx)
% 二维移动均值滤波器。
% 输入：x 为 MxN 二维数据；mt、mx 分别为时间方向和空间方向窗口长度。
% 输出：y 与 x 尺寸相同，每一点取其二维邻域内有效样点的平均。
% 边界处仅统计实际存在的点，不进行补零。

[M,N] = size(x);
rt = floor(mt/2);
```

```
rx = floor(mx/2);
y = zeros(M,N);

for i = 1:M
    for j = 1:N
        s = 0;
        c = 0;

        for p = i-rt:i+rt
            for q = j-rx:j+rx
                if p >= 1 && p <= M && q >= 1 && q <= N
                    s = s + x(p,q);
                    c = c + 1;
                end
            end
        end
        y(i,j) = s/c;
    end
end
end
```

zzh_tracefill.m

```
function y = zzh_tracefill(x,mis)
% 用相邻有效道插值恢复二维地震数据中的缺失道。
% 输入：x 为“采样点 x 道号”矩阵，mis 为缺失道号向量。
% 对内部缺道取左右最近有效道的平均；边界缺道复制唯一可用的一侧。

y = x;
[m,n] = size(x);

for k = 1:length(mis)
    j = mis(k);
    l = 0;
    r = 0;

    % 分别向左右搜索最近的有效道。
    for q = j-1:-1:1
        hit = 0;
        for p = 1:length(mis)
            if q == mis(p)
                hit = 1;
            end
        end
        if hit == 0
```

```
            l = q;
            break;
        end
    end

    for q = j+1:n
        hit = 0;
        for p = 1:length(mis)
            if q == mis(p)
                hit = 1;
            end
        end
        if hit == 0
            r = q;
            break;
        end
    end

    % 内部缺道取两侧平均，边界缺道复制可用的一侧。
    for i = 1:m
        if l > 0 && r > 0
            y(i,j) = (y(i,l)+y(i,r))/2;
        elseif l > 0
            y(i,j) = y(i,l);
        elseif r > 0
            y(i,j) = y(i,r);
        end
    end
end
end
```

zzh_DFT2.m

```
function F = zzh_DFT2(x)
% 利用二维 DFT 的可分离性计算二维频谱。
% 二维变换可分解为：先对每一列做一维 DFT，再对中间结果的每一行做 DFT。
% 这样可以复用自编 zzh_DFT，并清楚展示二维变换的实现原理。

[M,N] = size(x);
temp = zeros(M,N);
F = zeros(M,N);

% 第一阶段：沿第 1 维（时间/行方向）对每一列变换。
for j = 1:N
    col = zzh_DFT(x(:,j));
```

```
        temp(:,j) = col;
end


% 第二阶段：沿第 2 维（空间/列方向）对每一行变换。
for i = 1:M
        F(i,:) = zzh_DFT(temp(i,:)).';
end
end
```

zzh_shift2.m

```
function y = zzh_shift2(x)
% 将二维频谱的零频分量移动到图像中心。
% 作用与 fftshift 类似：行方向循环移动 floor(M/2)，
% 列方向循环移动 floor(N/2)，便于观察 FK 谱的对称结构。

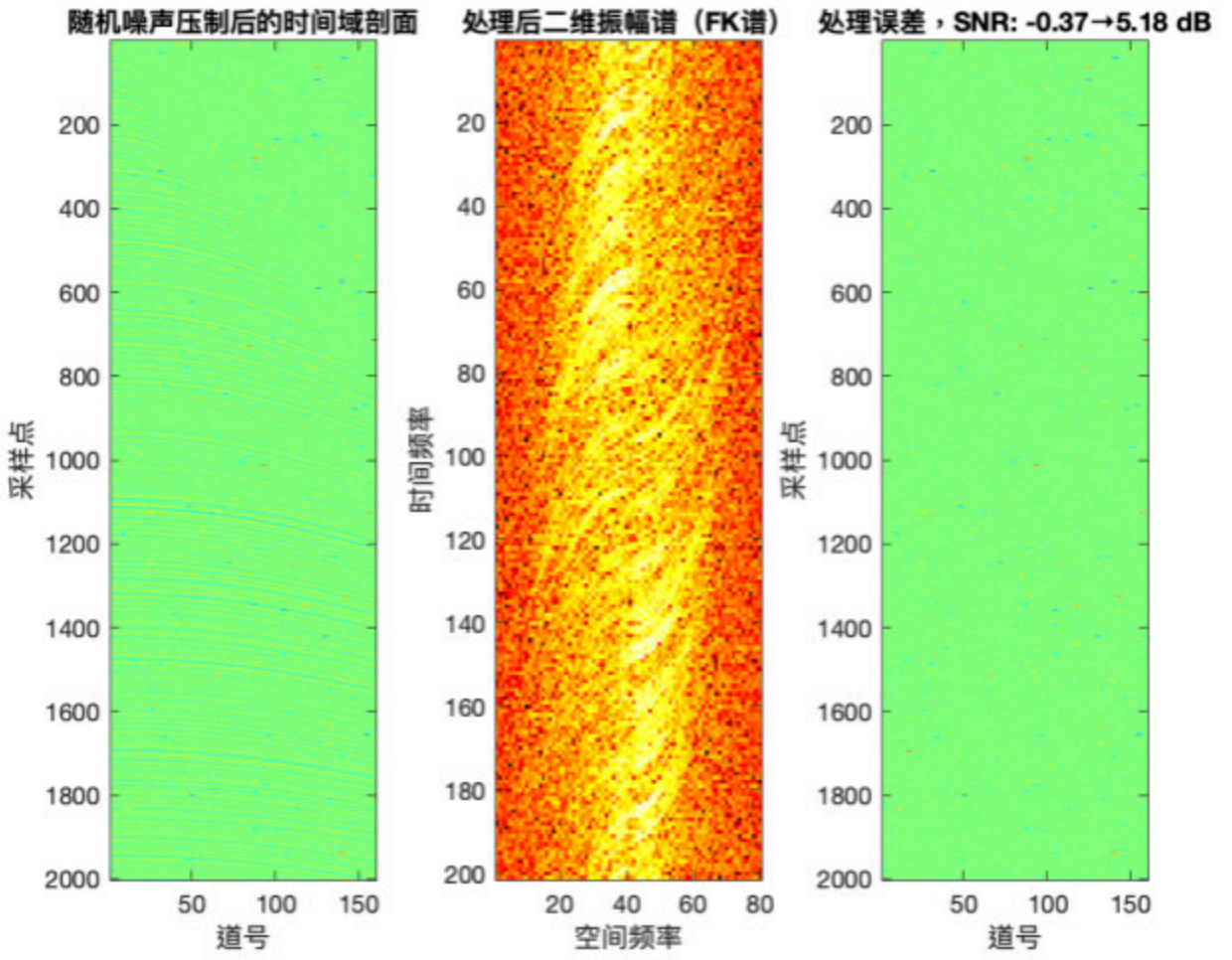
[M,N] = size(x);
y = zeros(M,N);
rm = floor(M/2);
rn = floor(N/2);

for i = 1:M
    for j = 1:N
        ii = i-1+rm;
        if ii >= M
            ii = ii-M;
        end

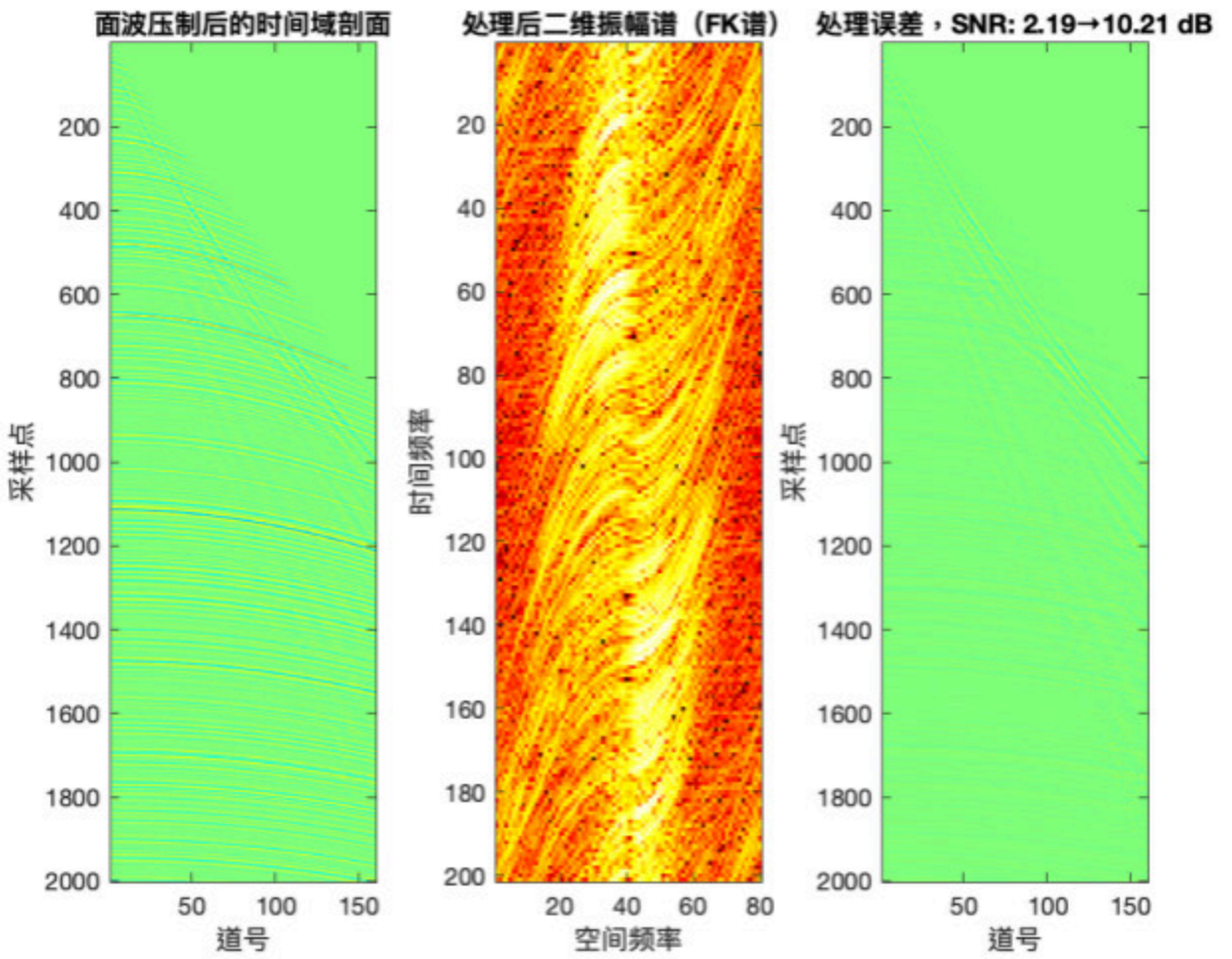
        jj = j-1+rn;
        if jj >= N
            jj = jj-N;
        end

        y(i,j) = x(ii+1,jj+1);
    end
end
end
```

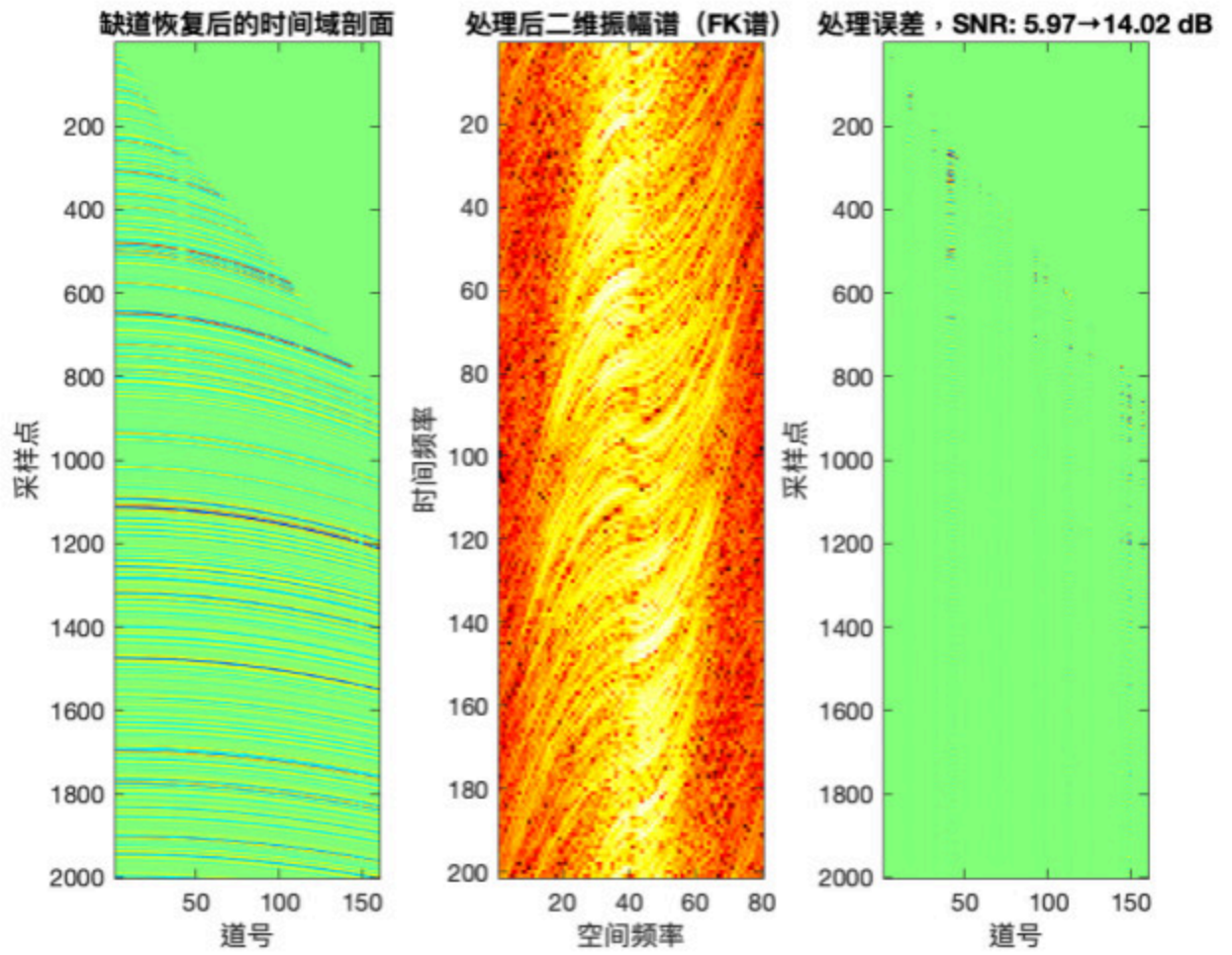
【运行结果与分析】



二维随机噪声压制结果



面波压制结果



缺道恢复结果

二维随机噪声处理后，局部不相干波动减弱，反射同相轴连续性增强；面波处理后低频强能量受到压制，但移动平均高通也可能削弱部分缓慢变化的有效成分；缺失道经相邻有效道插值后恢复了空间连续性，在地层横向变化平缓时效果较好。三项处理后的 SNR 均提高，说明误差总体减小。对于连续多道缺失或构造横向变化剧烈的情况，简单线性插值会存在局限，可进一步考虑多道加权或 FK 域重建。